Réseaux de Neurones Convolutifs

Apprentissage Profond

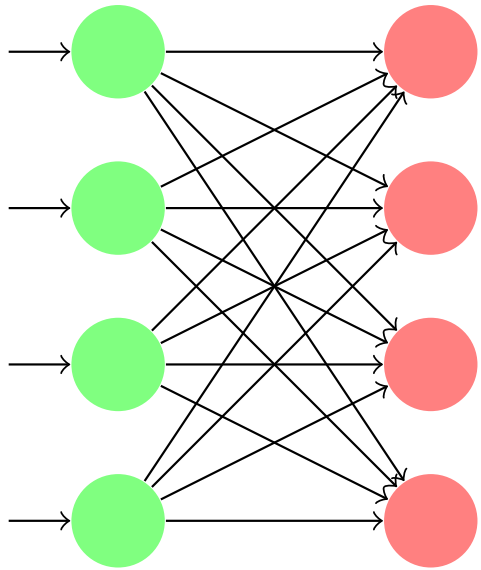
A. Carlier

2026

Exemple de la classification d'images sur ImageNet

Images de taille $224 \times 224 \times 3$, 1000 classes :





$224 \times 224 \times 3$ pixels

$224 \times 224 \times 3$ neurones

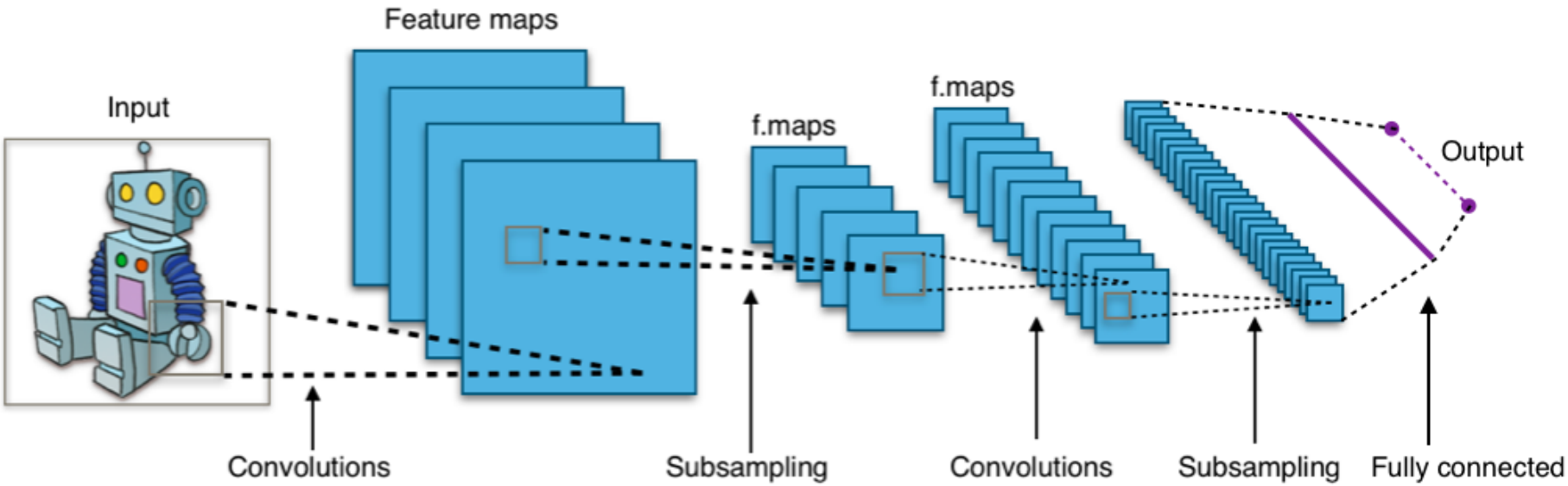
1000 neurones

Un perceptron monocouche nécessite $224 \times 224 \times 3 \times 1000 + 1000$ paramètres, soit 150 millions de paramètres !

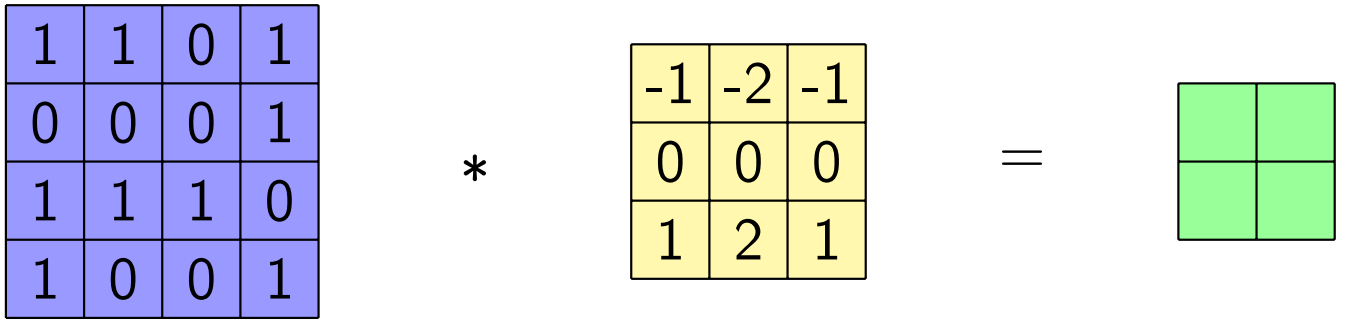
Plan du cours

1. Couches de convolution
2. Architectures convolutives
3. Visualisation
4. Transfert d'apprentissage
5. Inception et ResNet
6. Segmentation

Architecture classique d'un réseau de neurones convolutif



Convolution

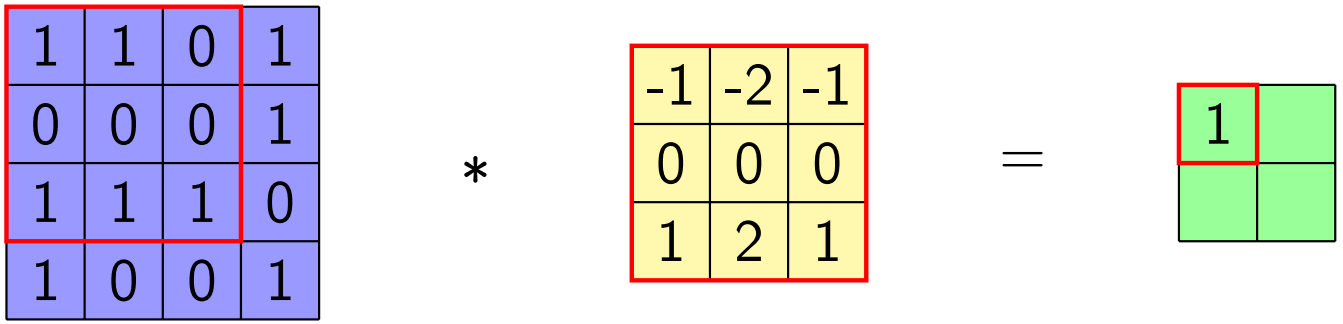


Image

Filtre 3×3
$f = 3$

Réponse

Convolution



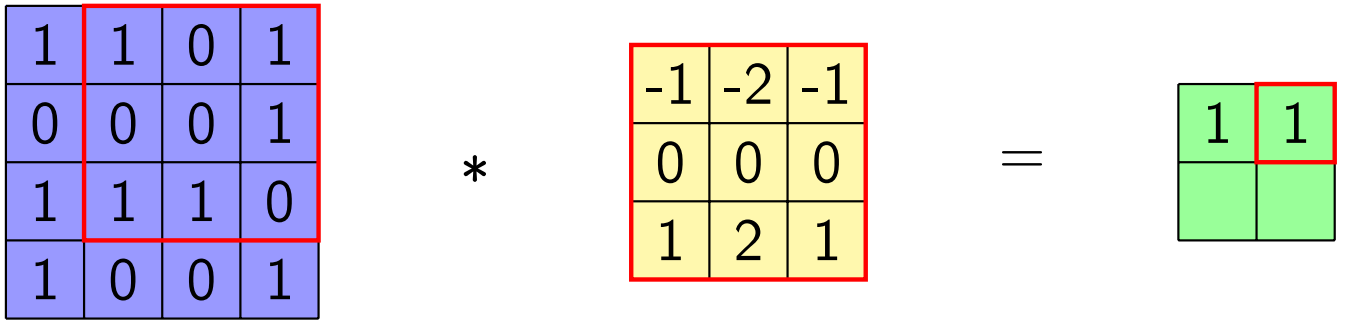
Image

Filtre 3×3
$f = 3$

Réponse

$$1 * -1 + 1 * -2 + 0 * -1 + 0 * 0 + 0 * 0 + 0 * 0 + 1 * 1 + 1 * 2 + 1 * 1 = 1$$

Convolution

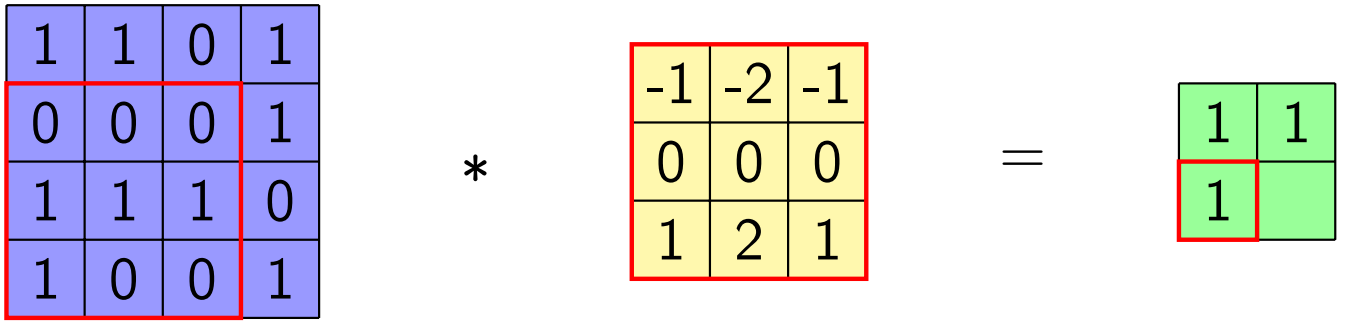


Image

Filtre 3×3
$f = 3$

Réponse

Convolution

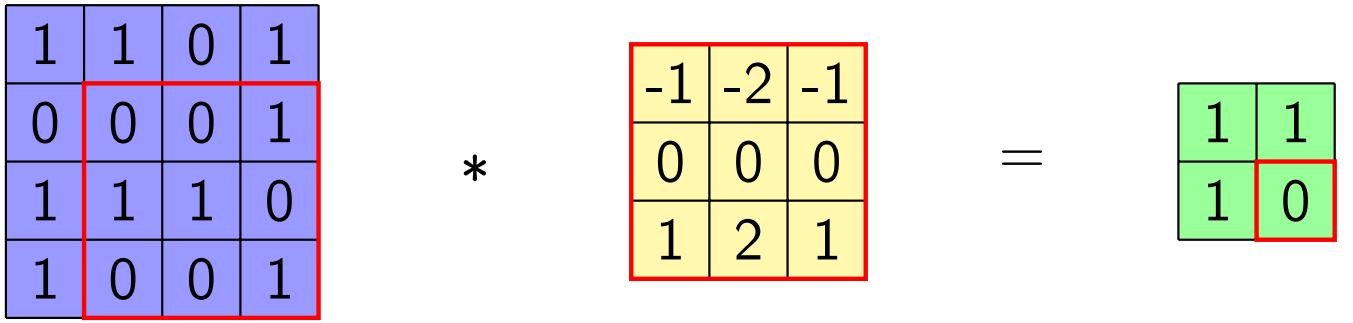


Image

Filtre 3×3
$f = 3$

Réponse

Convolution

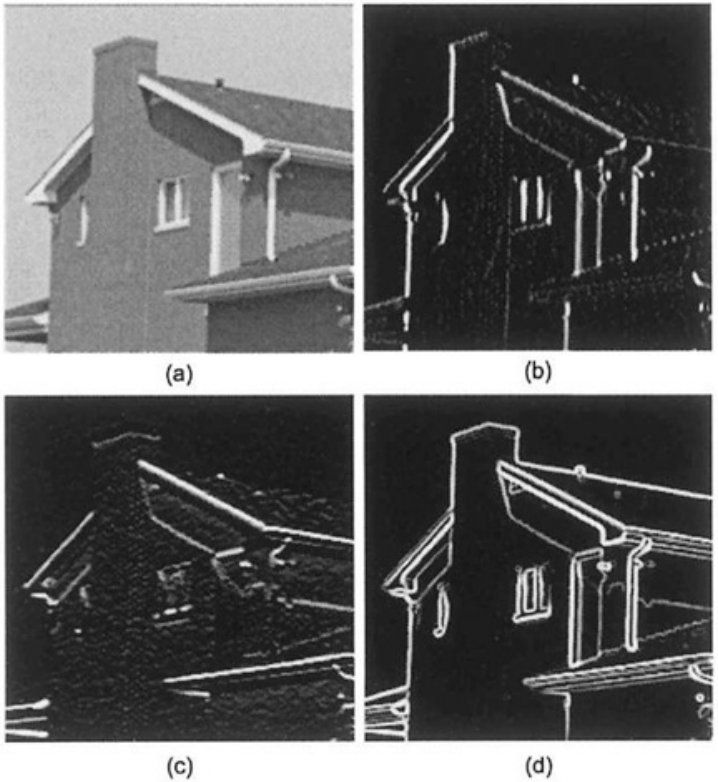


Image

Filtre 3×3
$f = 3$

Réponse

La convolution en Vision par Ordinateur



-1	0	+1
-2	0	+2
-1	0	+1

Gx

+1	+2	+1
0	0	0
-1	-2	-1

Gy

- Les filtres (ou noyaux) de convolution sont utilisés depuis longtemps pour détecter des motifs dans les images, comme par exemple les contours (ici, filtres de Sobel)
- un pixel blanc indique une réponse élevée du filtre, c'est-à-dire un pixel situé sur le contour d'un objet, avec un fort gradient local.

Un point sur les dimensions des tenseurs

Étant donnés :

- une image I en niveaux de gris (un seul canal couleur) de dimension $w \times h$,
- un filtre K de dimension f,

Alors la dimension de la réponse I $\circledast$ K de l'image I au filtre K est $(w - f + 1) \times (h - f + 1)$

Convolution (2D) de volumes

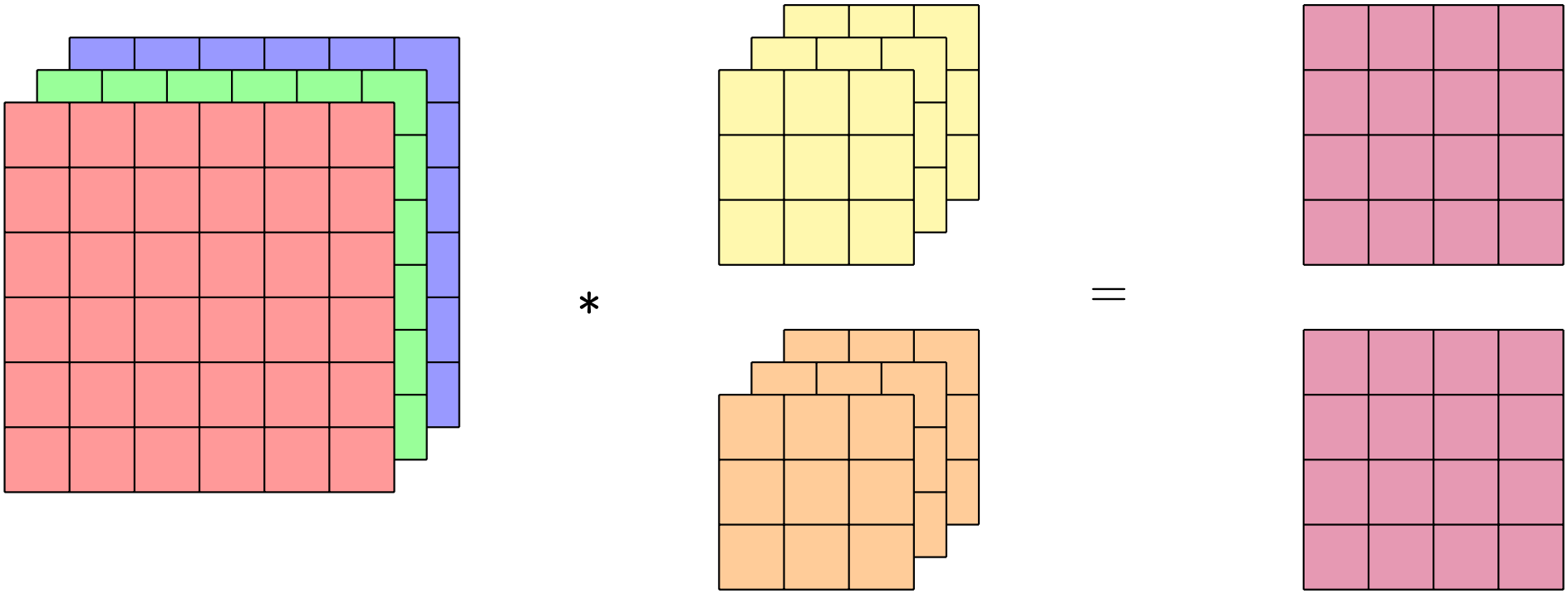


Image
$6 \times 6 \times \mathbf{3}$

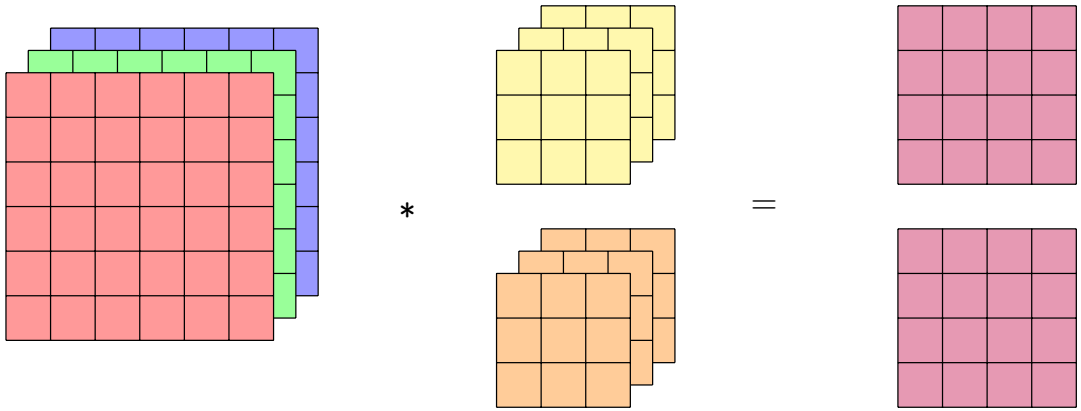
Filtres
$3 \times 3 \times \mathbf{3} \times 2$

Réponse
$4 \times 4 \times 2$

$\rightarrow$ en Pytorch : Conv2d(*in_channels*,*out_channels*,*kernel_size*)

Le nombre de canaux de l'image d'entrée et la profondeur des filtres de convolution sont nécessairement identiques.

Nombre de paramètres d'une couche de convolution

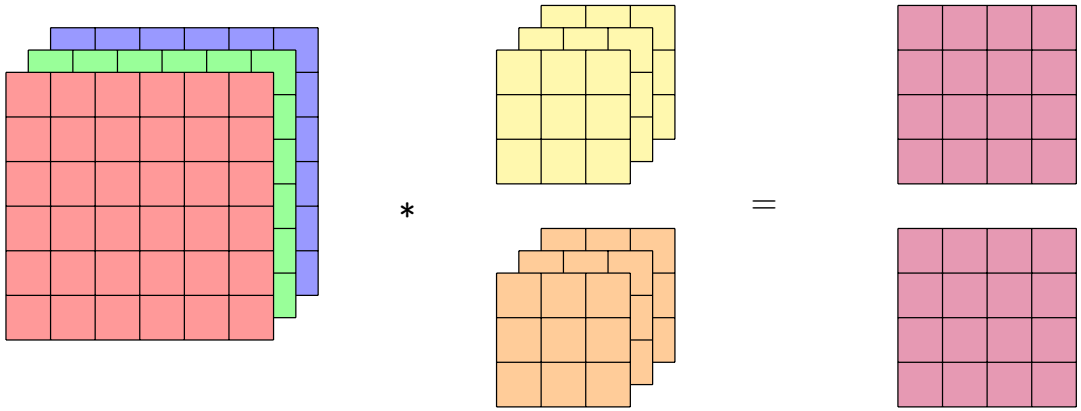


Il y a 2 types de paramètres dans une couche de convolution :

- Les **coefficients des filtres de convolution** : il y en a donc $f \times f \times$ #canaux $\times$ #filtres
- Les **biais** additionnés à la réponse des filtres de convolution, avant l'application de la fonction d'activation. Il y a exactement un biais par filtre de convolution.

Ainsi dans l'exemple ci-dessus, il y a 3x3x3x2 + 2 = 56 paramètres.

Nombre d'opérations d'une couche de convolution



Il est intéressant de compter le nombre d'opérations d'un réseau de neurones pour caractériser sa complexité, et les ressources nécessaires à son exécution. On s'intéresse principalement aux additions et aux multiplications.

Ici, chaque élément de la réponse nécessite :

- ($f \times f \times$ #canaux) multiplications
- ($f \times f \times$ #canaux - 1) additions entre les valeurs multipliées
- 1 addition supplémentaire pour l'ajout du biais

Ainsi dans l'exemple ci-dessus, il y a (4x4x2) x (3x3x3x2) = 1728 opérations.

Padding

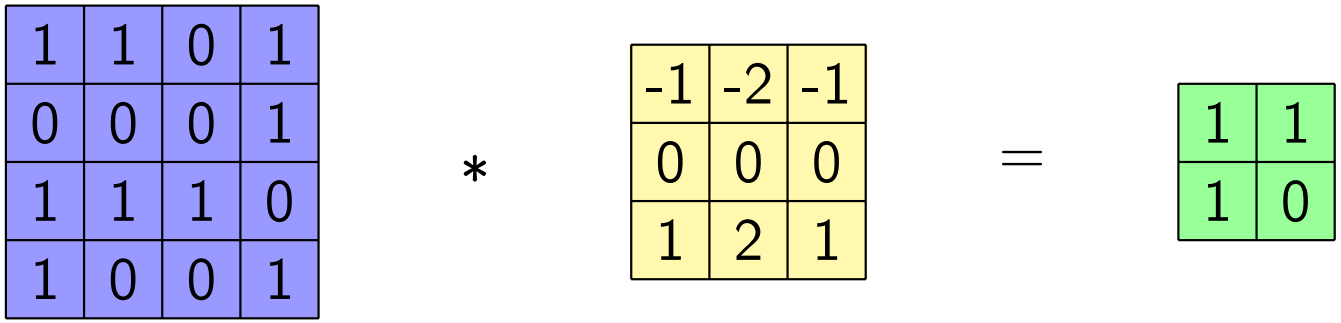


Image Filtre Réponse

Padding

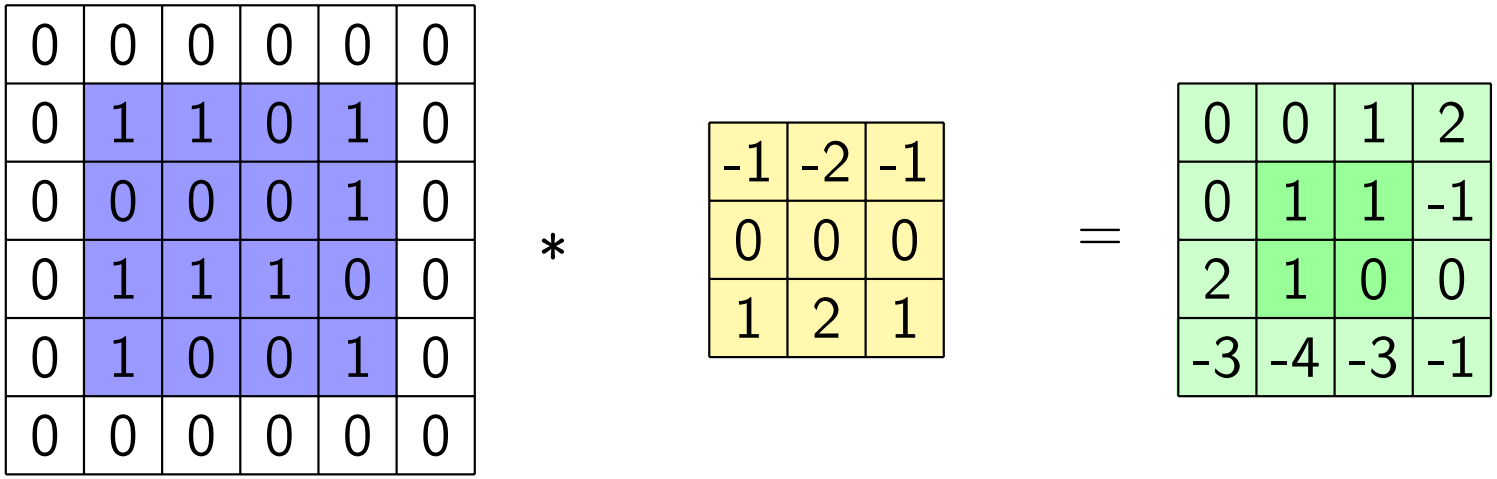


Image
$p = 1$

Filtre

Réponse

- Ajout de zéros (*zero-padding*) aux bords.
- Permet par exemple d'obtenir une réponse de même dimension que l'image d'entrée (convolution *same*), ce qui facilite le chaînage des couches de convolution dans un réseau de neurones.

Impact sur la dimension des tenseurs

Étant donnés :

- une image I en niveaux de gris (un seul canal couleur) de dimension $w \times h$,
- un filtre K de dimension f, avec un *padding* p

Alors la dimension de la réponse $I \circledast K$ de l'image I au filtre K est $(w + 2p - f + 1) \times (h + 2p - f + 1)$

Stride

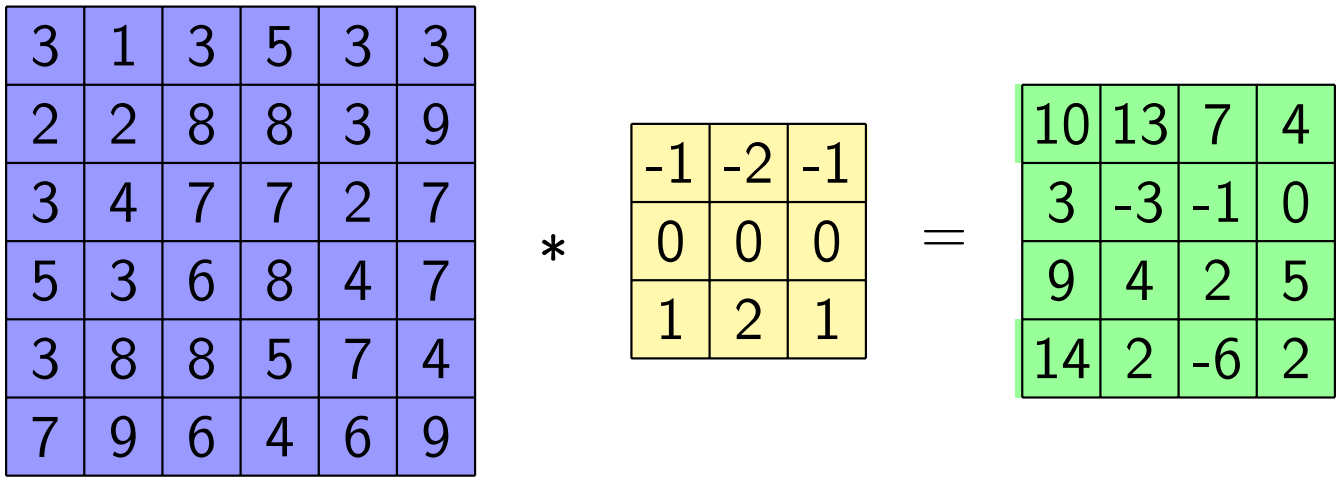


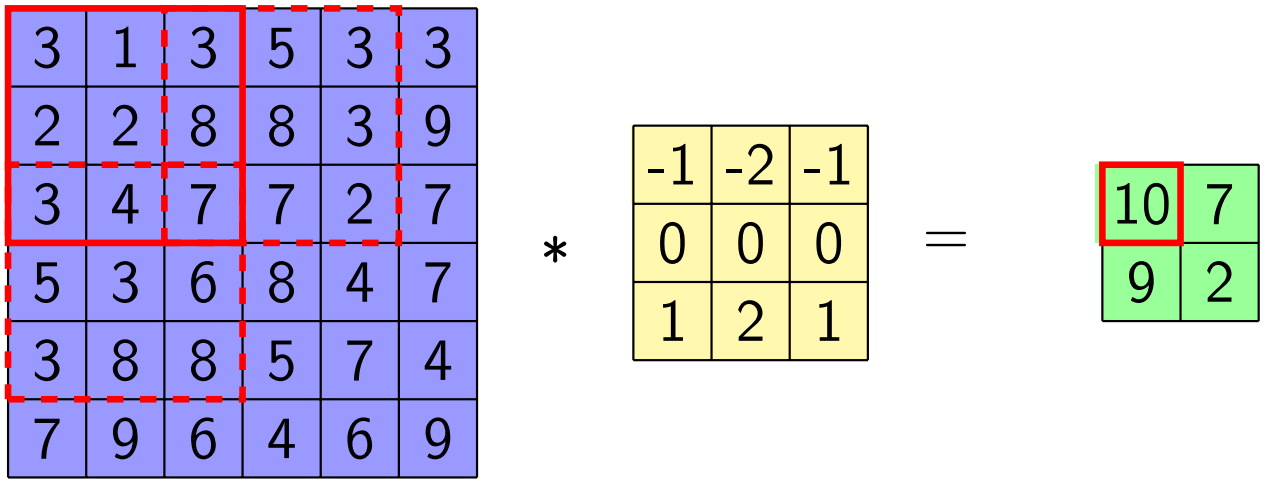
Image
$p = 0$

Filtre
$f = 3$

$stride = 1$

Réponse

Stride



- Permet de **réduire la dimension** des tenseurs, en limitant la perte d"information du fait qu'un même coefficient influence plusieurs éléments de la réponse au filtre de convolution.

Impact sur la dimension des tenseurs

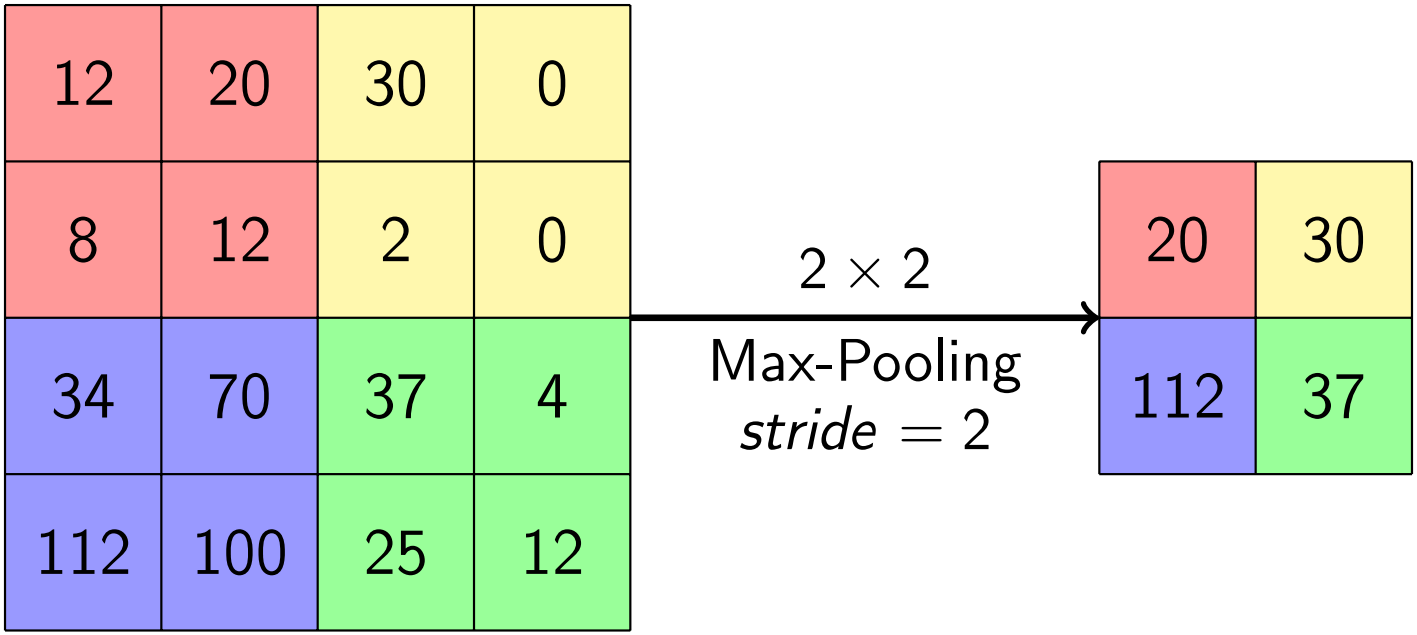
Étant donnés :

- une image I en niveaux de gris (un seul canal couleur) de dimension $w \times h$,
- un filtre K de dimension f, avec un *padding* p et un *stride* s

Alors la dimension de la réponse I $\circledast$ K de l'image I au filtre K est :

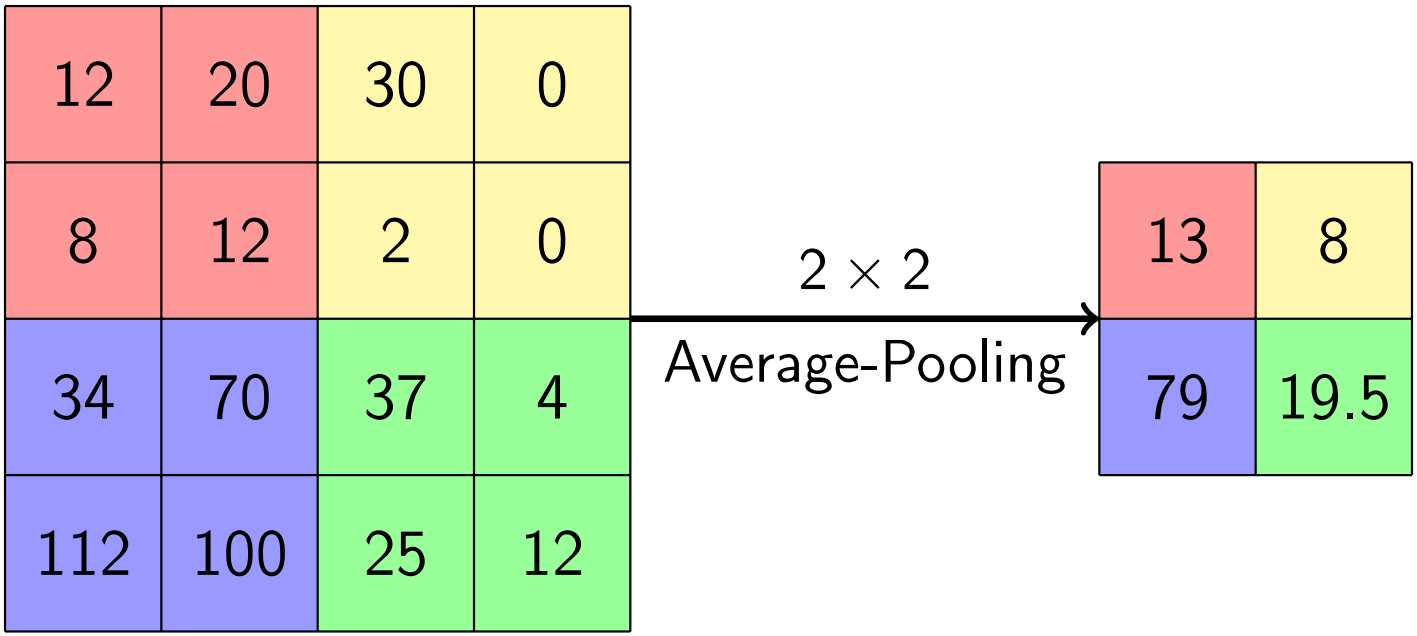
$$\lfloor \frac{w + 2p - f}{s} + 1 \rfloor \times \lfloor \frac{h + 2p - f}{s} + 1 \rfloor \quad (1)$$

Couche de Pooling



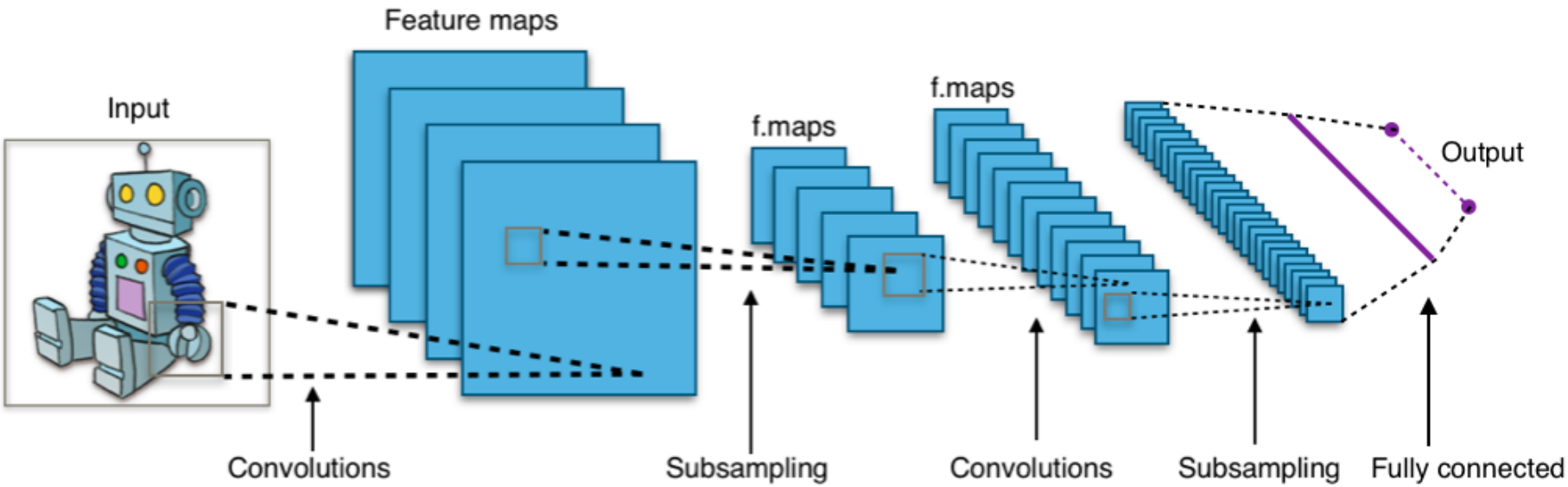
- Permet de **réduire la dimension** des tenseurs, pour contrebalancer la multiplication des réponses aux filtres de convolution.
- **Préserve les hautes réponses** des filtre de convolution.
- Introduit une **invariance à la translation**.
- **Pas de paramètres** à apprendre !

Couche de Pooling



- Alternativement, on peut aussi moyenner les valeurs plutôt qu'en conserver le maximum (on parle d'**Average-Pooling**).
- Les architectures classiques privilégient cependant le **Max-Pooling**.

Architecture classique d'un réseau de neurones convolutif



On trouve 3 types de couches dans un réseau de neurones convolutif typique :

- Des couches de **convolution**, combinées à des couches de **pooling** dans les premières couches du réseau.
- Des couches **complètement connectées** dans les dernières couches du réseau.

Intérêt des réseaux de neurones convolutifs

Partage de paramètres :

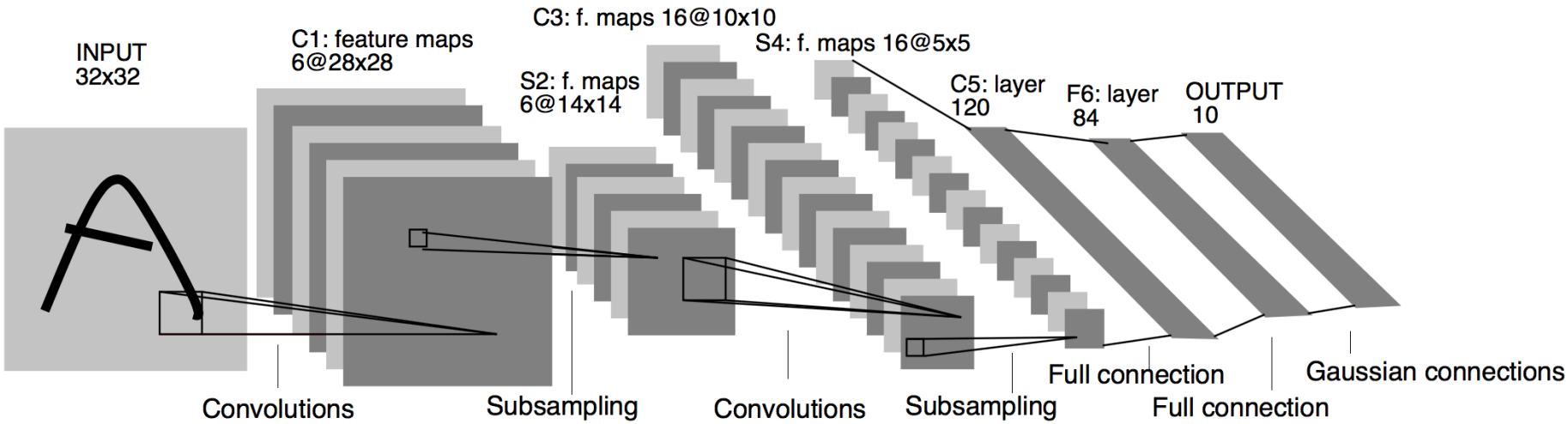
- Une couche de convolution est équivalente à une couche complètement connectée dans laquelle certains poids synaptiques sont partagés, et dont la majorité est à 0.
- Un même exemple de la base d'apprentissage permet de modifier ces poids (les coefficients de convolution) à de multiples reprises.

Ceci résulte en un nombre de paramètres bien plus faible pour un réseau convolutif que pour un réseau complètement connecté.

Plan du cours

1. Couches de convolution
2. Architectures convolutives
3. Visualisation
4. Transfert d'apprentissage
5. Inception et ResNet
6. Segmentation

Un pionnier : LeNet (1998)

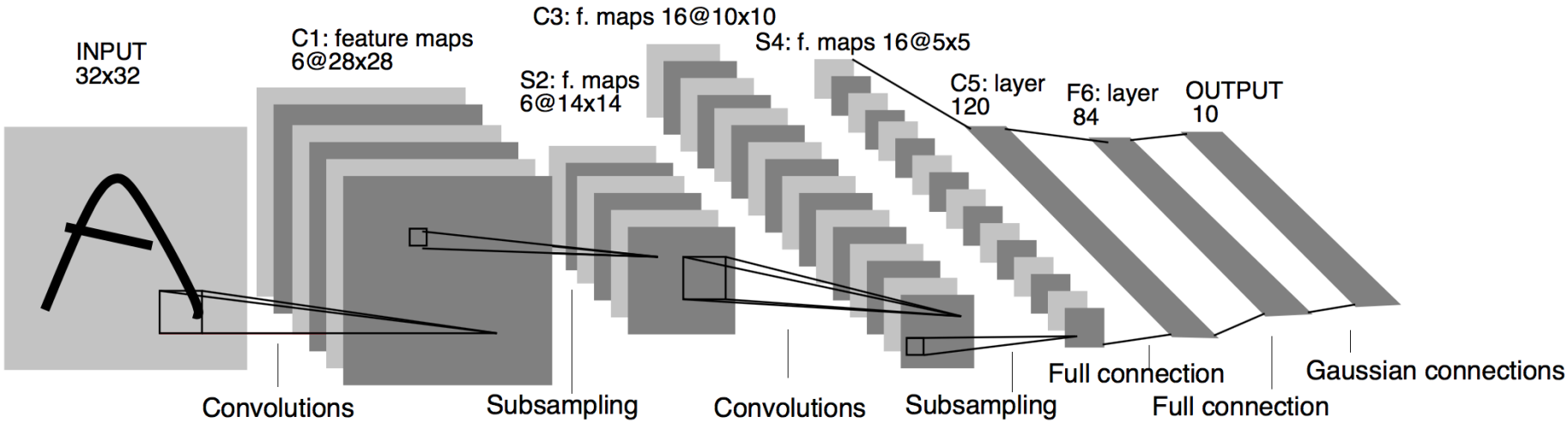


- $\simeq$ 60k paramètres
- 2 à 3 jours d'entraînement pour 20 *epochs* sur MNIST (en 1998 !).
- Fonctions d'activation sigmoïdes en majorité.

Visualisation : `https://adamharley.com/nn_vis/cnn/2d.html`

[LeCun et al.] Gradient-based learning applied to document recognition.

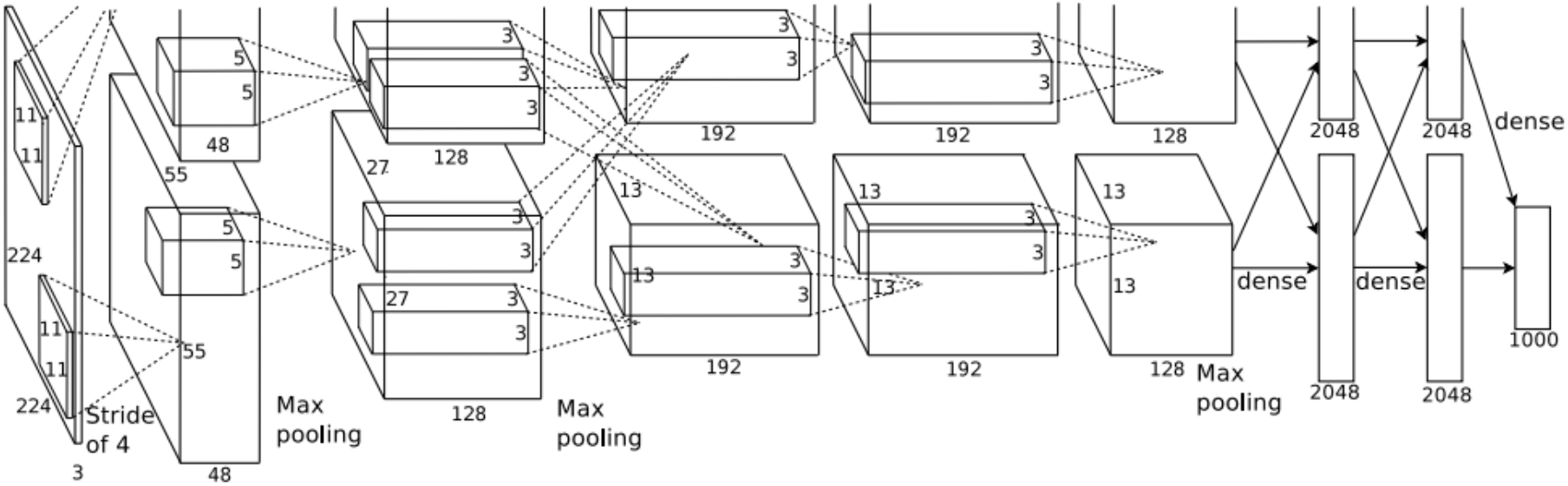
Question



Nombre de paramètres ? Nombre d'opérations à l'inférence ?

Question subsidiaire : poids en mémoire des calculs intermédiaires ?

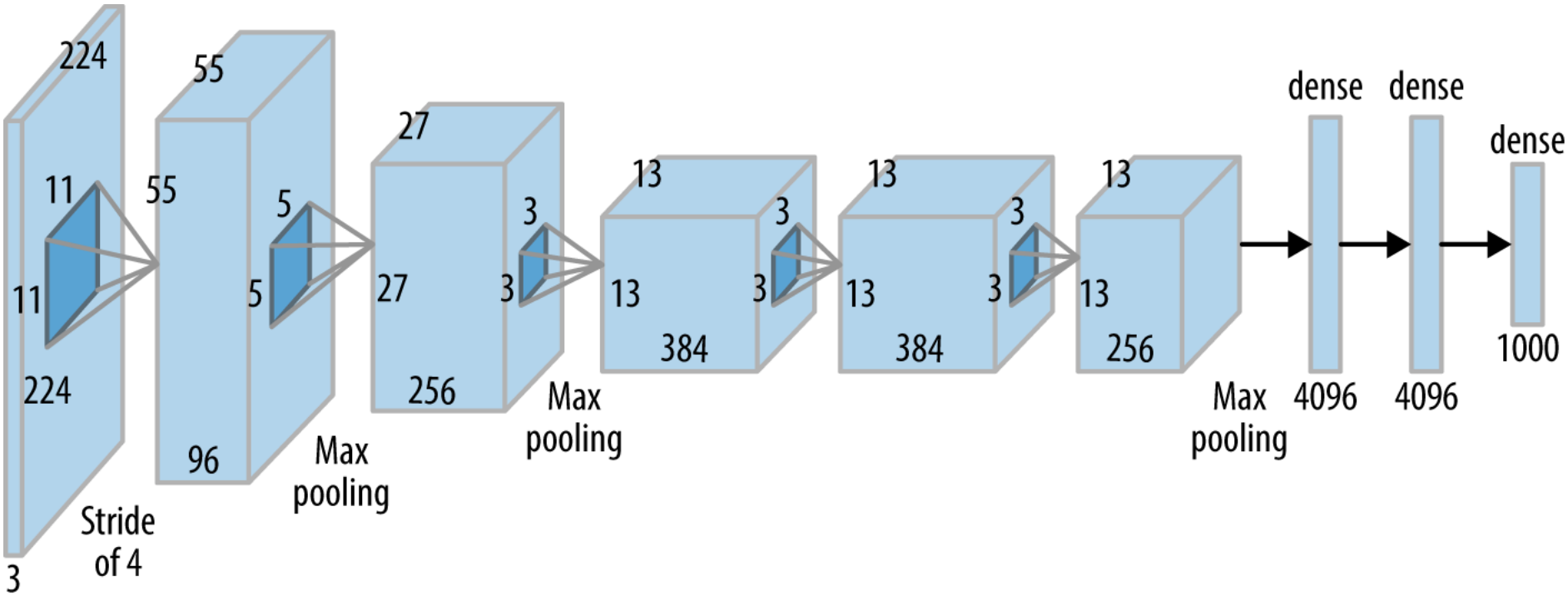
La bascule : AlexNet (2012)



- $\simeq$ 60M paramètres, 8 couches
- l'architecture est conçue pour être implémentée sur 2 GPUs.
- introduit l'usage de la fonction `reLU` comme un standard pour l'entraînement de réseaux profonds.

[Krizhevsky et al.] ImageNet Classification with Deep Convolutional Neural Networks.

La bascule : AlexNet (2012)

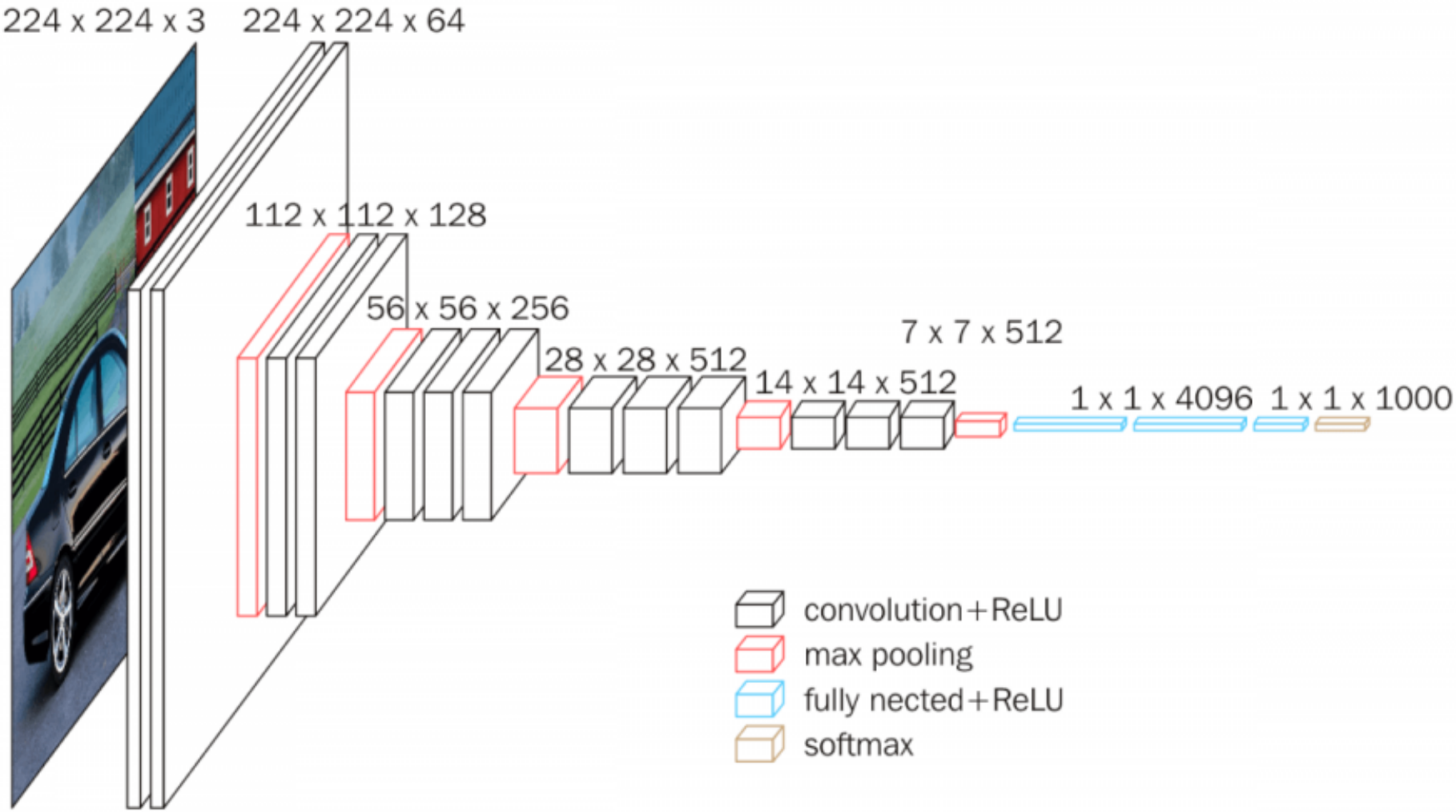


Observations :

- Diminution progressive de la taille des filtres ($11 \rightarrow 5 \rightarrow 3$)
- Diminution progressive de la taille de l'image ($224 \rightarrow 55 \rightarrow 27 \rightarrow 13$)
- Augmentation progressive du nombre de filtres ($96 \rightarrow 256 \rightarrow 384$)
- *Stride* puis *Max Pooling*

[Krizhevsky et al.] ImageNet Classification with Deep Convolutional Neural Networks.

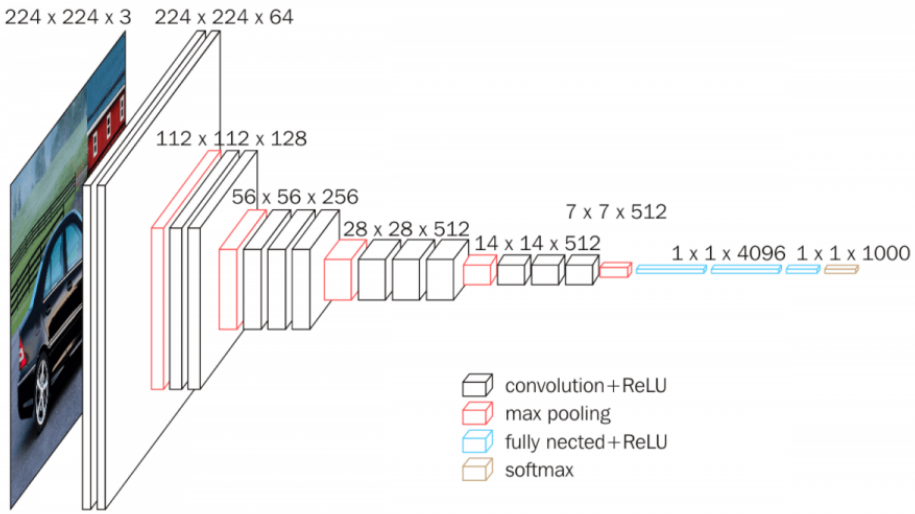
Une version "simplifiée" : VGG-16 (2014)



$\simeq$ 138M paramètres, 16 couches dans sa version standard.

[Simonyan et Zisserman] Very Deep Convolutional Networks for Large-Scale Image Recognition

Une version "simplifiée" : VGG-16 (2014)

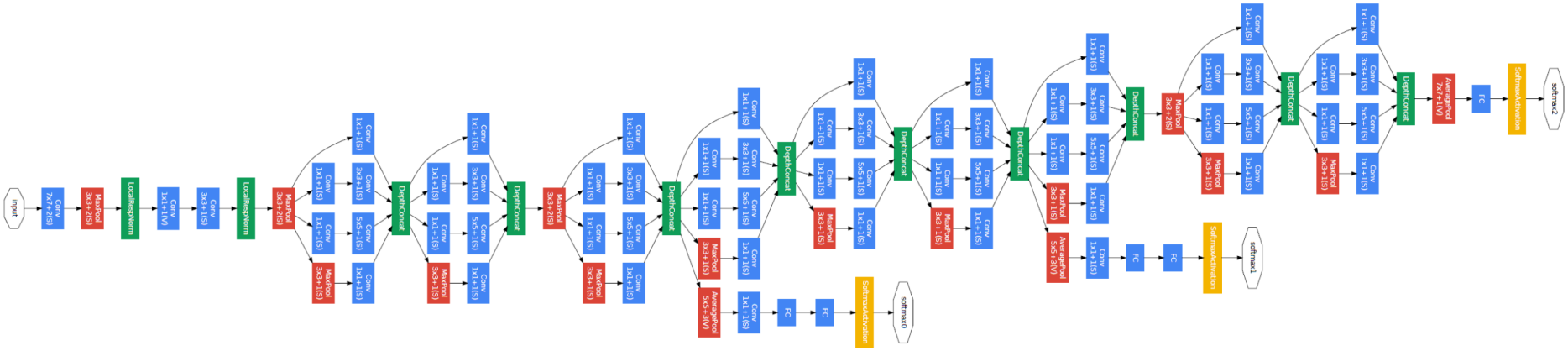


Objectif : étudier l'impact de la profondeur sur les performances du réseau.
$\rightarrow$ Pour cela, les auteurs ont rendu l'architecture du réseau très régulière :

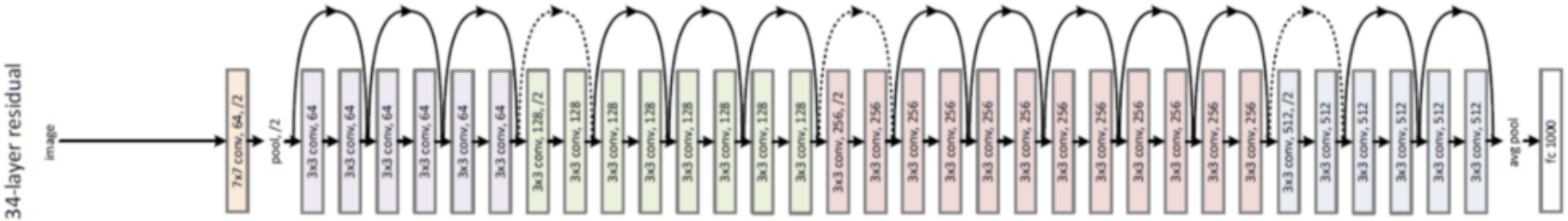
- Utilisation systématique de convolutions 3×3
- Reprise des grandes caractéristiques d'`AlexNet`, en les régularisant :
  - Diminution progressive de la taille de l'image ($224 \rightarrow 112 \rightarrow 56$ ...)
  - Augmentation progressive du nombre de filtres ($64 \rightarrow 128 \rightarrow 256$...)

[Simonyan et Zisserman] Very Deep Convolutional Networks for Large-Scale Image Recognition

Architectures plus avancées

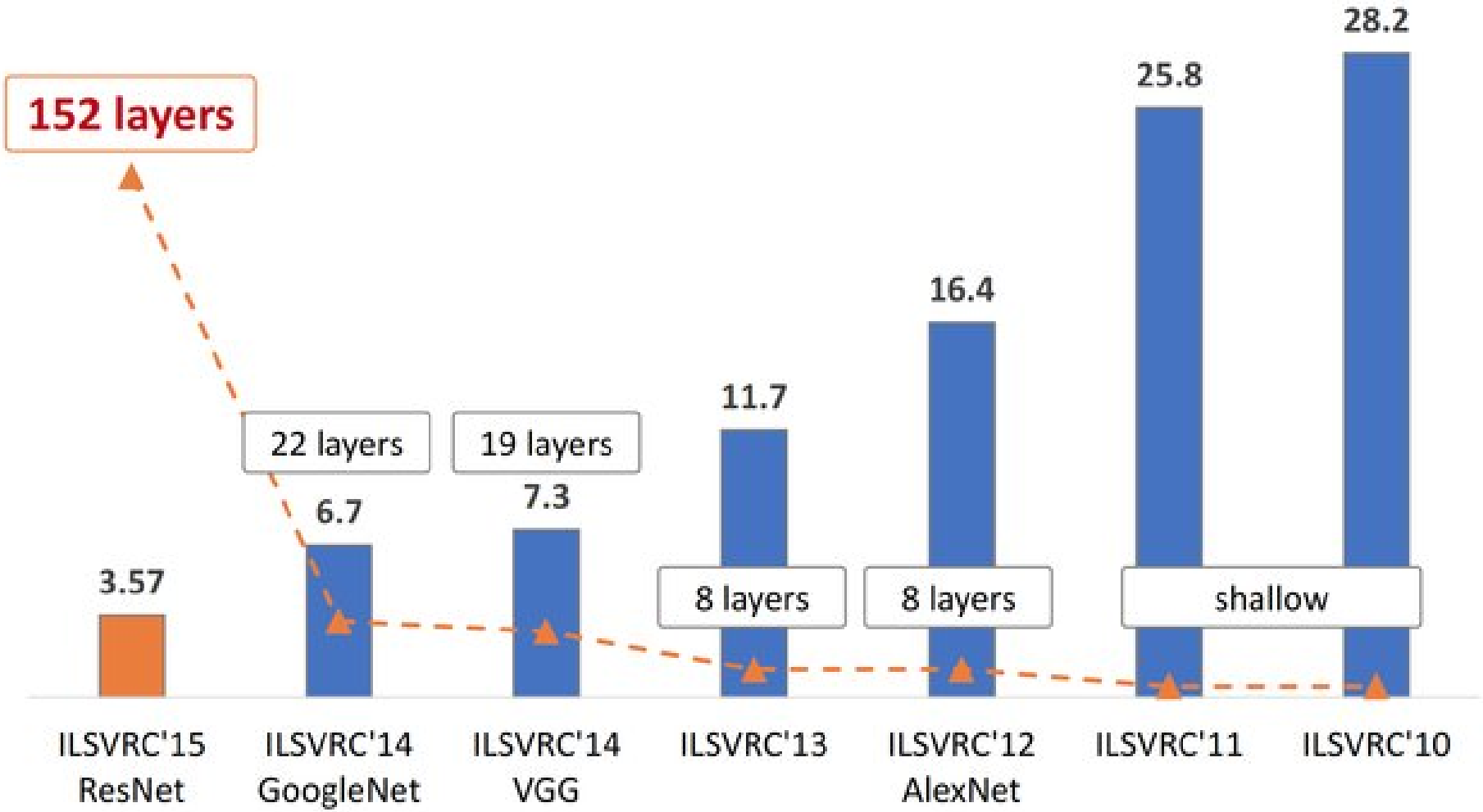


[Szegedy et al.] Going deeper with convolutions.

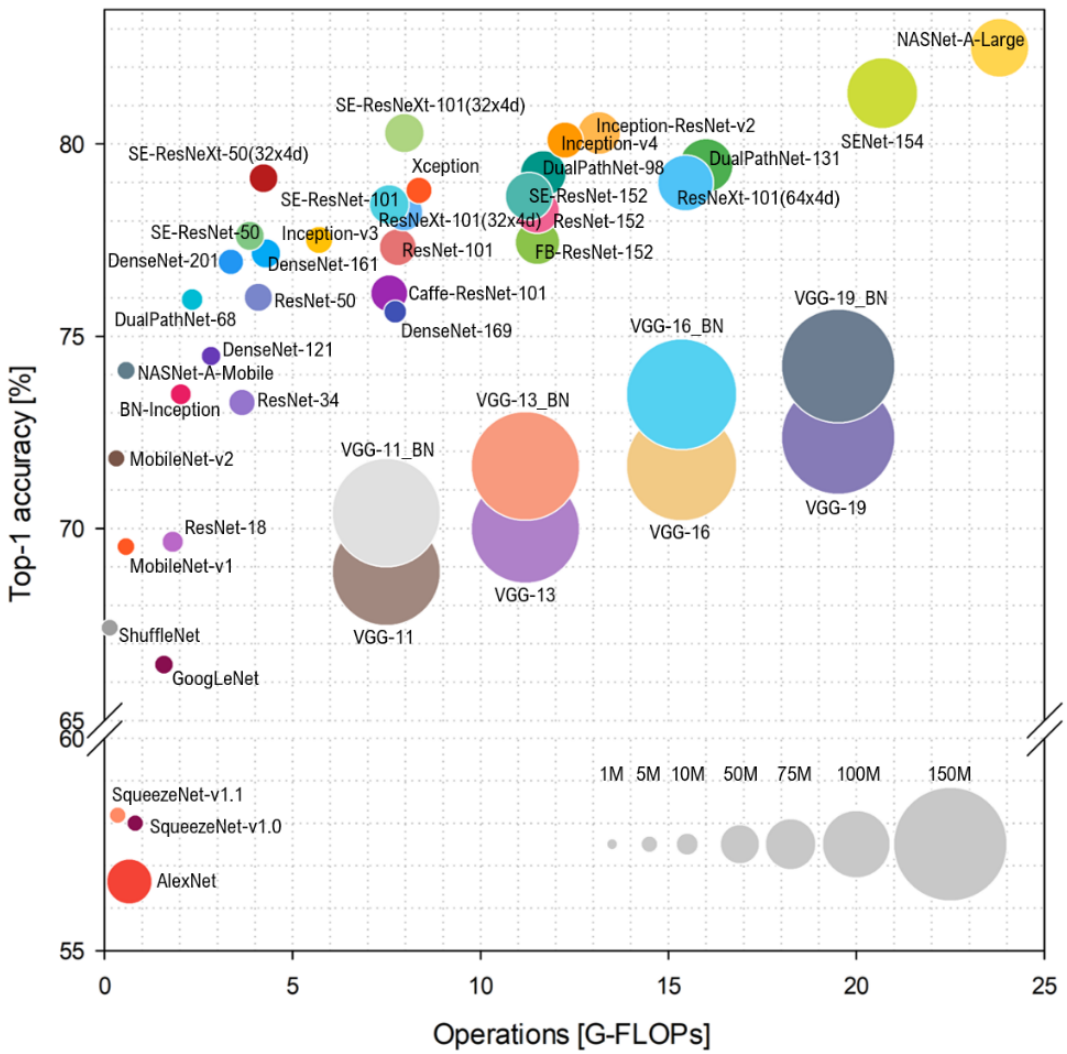


[He et al.] Deep Residual Learning for Image Recognition

2015 : fin du challenge ImageNet pour la classification d'image



Depuis 2015



[Bianco et al.] Benchmark Analysis of Representative Deep Neural Network Architectures

Plan du cours

1. Couches de convolution
2. Architectures convolutives
3. Visualisation
4. Transfert d'apprentissage
5. Inception et ResNet
6. Segmentation

Apprentissage par représentation

X Y

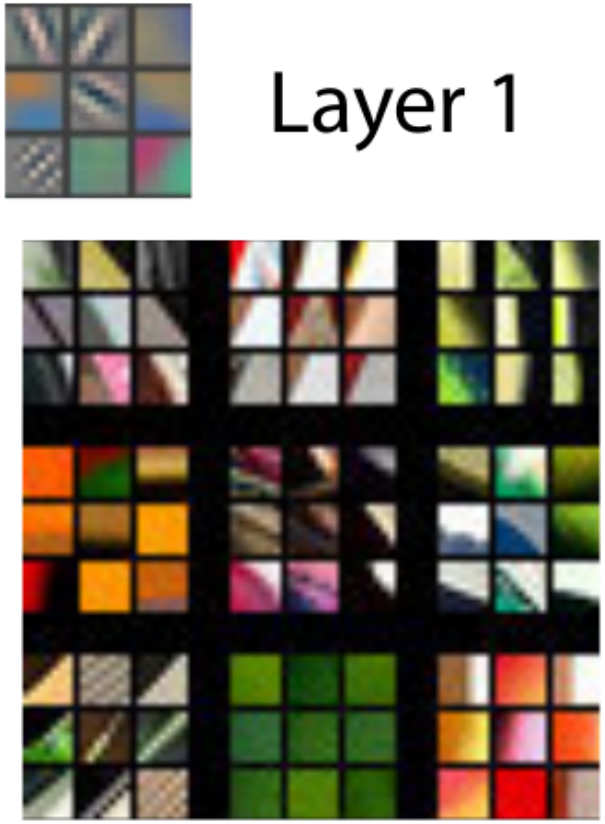


$\xrightarrow{h}$ "personne"

$\xrightarrow{f}$ $\boxed{f(X)}$ $\xrightarrow{h'}$

Caractéristiques

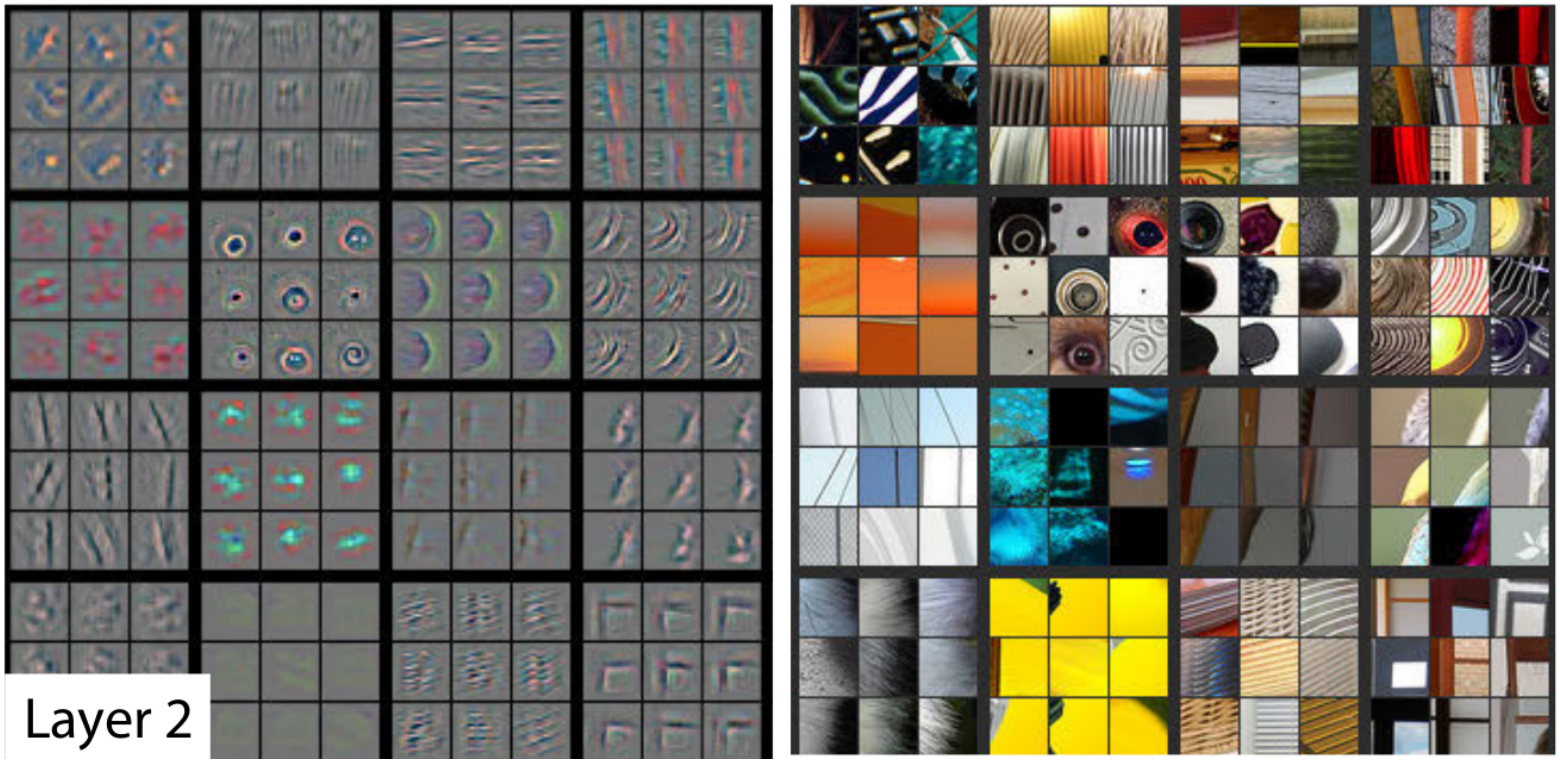
Qu'apprennent les réseaux convolutifs ?



Exemple de filtres appris sur la première couche d'un réseau (proche d'AlexNet), et pour chaque filtre, énumération des 9 patches d'images occasionnant la plus forte activation de ces filtres.

[Zeiler et al.] Visualizing and understanding convolutional networks.

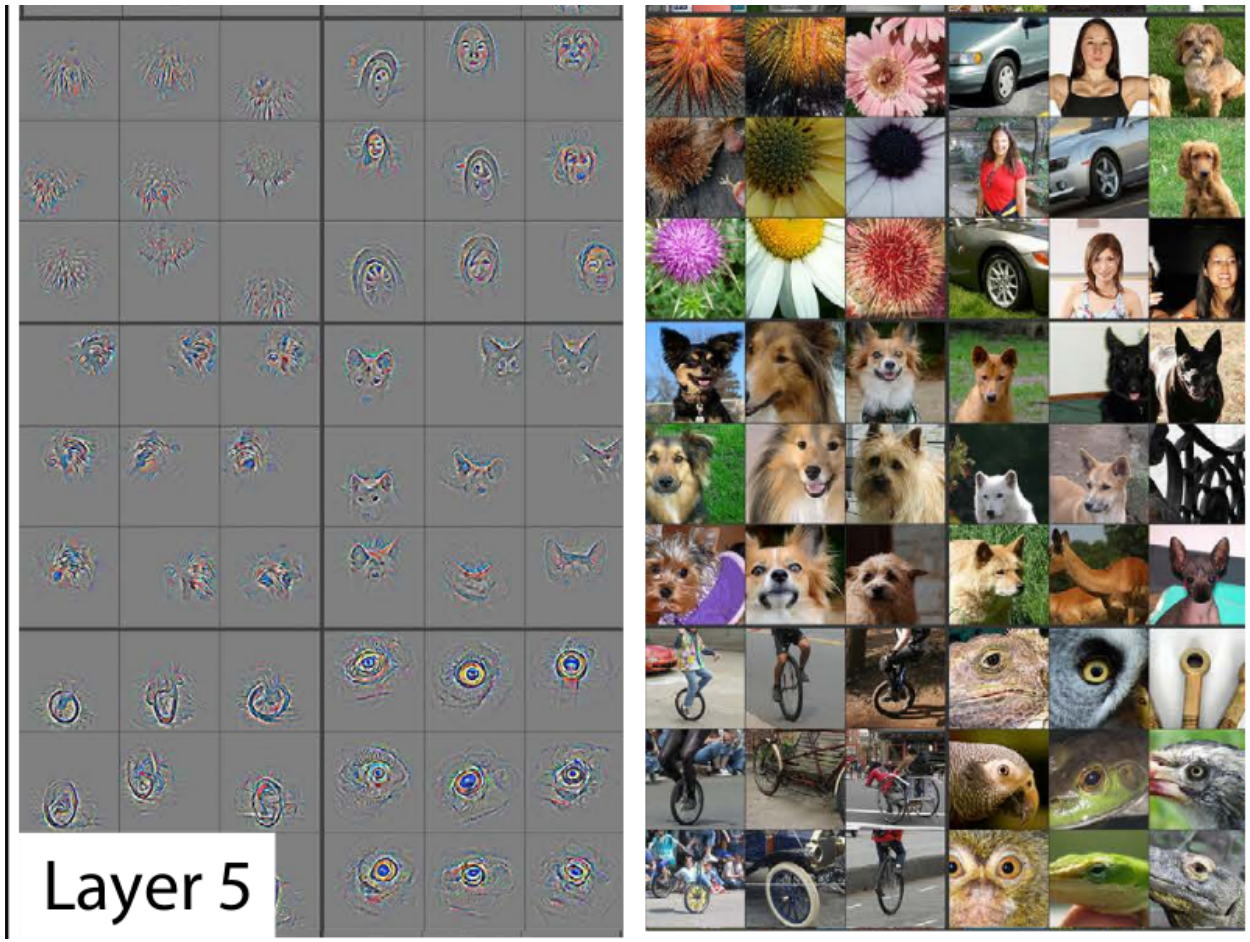
Qu'apprennent les réseaux convolutifs ?



Même visualisation que précédemment, mais pour les filtres de la deuxième couche. A noter que les filtres de la partie gauche ne sont pas présents tels quels dans le réseau, mais que cette représentation visuelle est reconstruite.

[Zeiler et al.] Visualizing and understanding convolutional networks.

Qu'apprennent les réseaux convolutifs ?



Les motifs détectés sont d'un plus haut niveau sémantique à mesure que l'on progresse dans les couches du réseau.

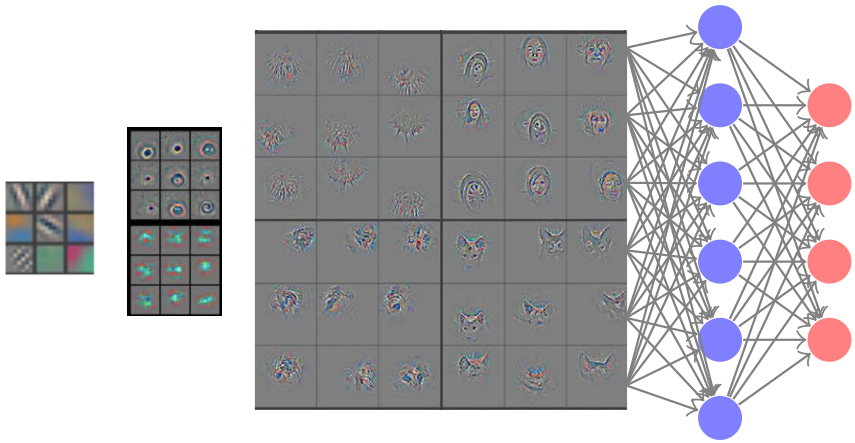
[Zeiler et al.] Visualizing and understanding convolutional networks.

Une manière d'interpréter les CNN

X $\quad f(X)$ $\quad h'$ $\quad Y$

Extraction de caractéristiques — Prédicteur





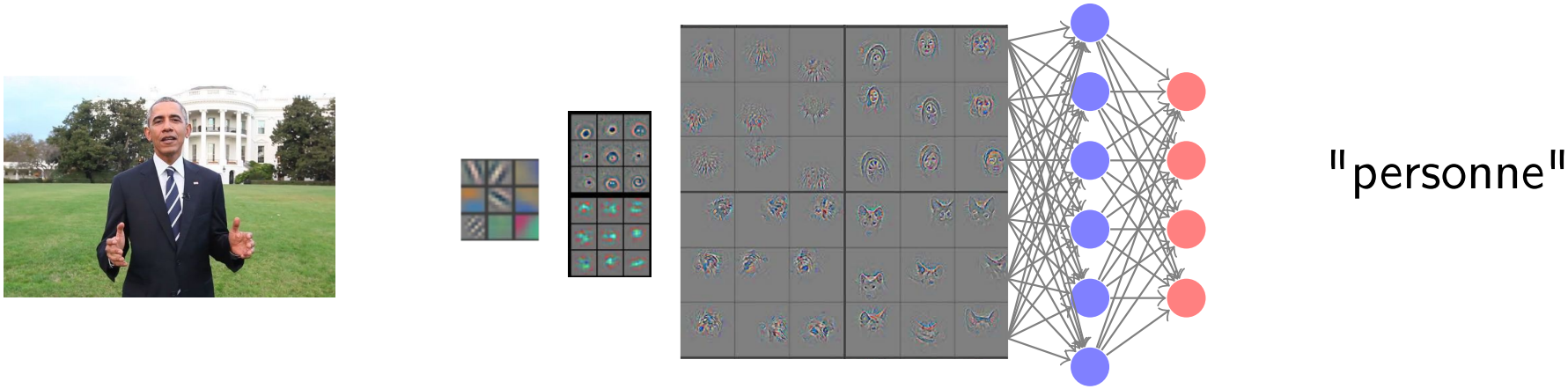
"personne"

On peut interpréter un CNN au regard de l'apprentissage par représentation : la partie convolutive est un extracteur de caractéristiques $f(X)$, et les couches denses de fin constituent notre prédicteur h'. L'apprentissage profond permet ainsi d'apprendre l'extracteur de caractéristiques en plus du prédicteur !

Plan du cours

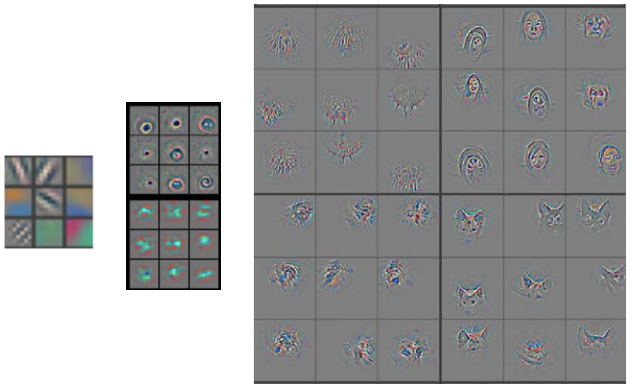
1. Couches de convolution
2. Architectures convolutives
3. Visualisation
4. Transfert d'apprentissage
5. Inception et ResNet
6. Segmentation

Transfert d'apprentissage



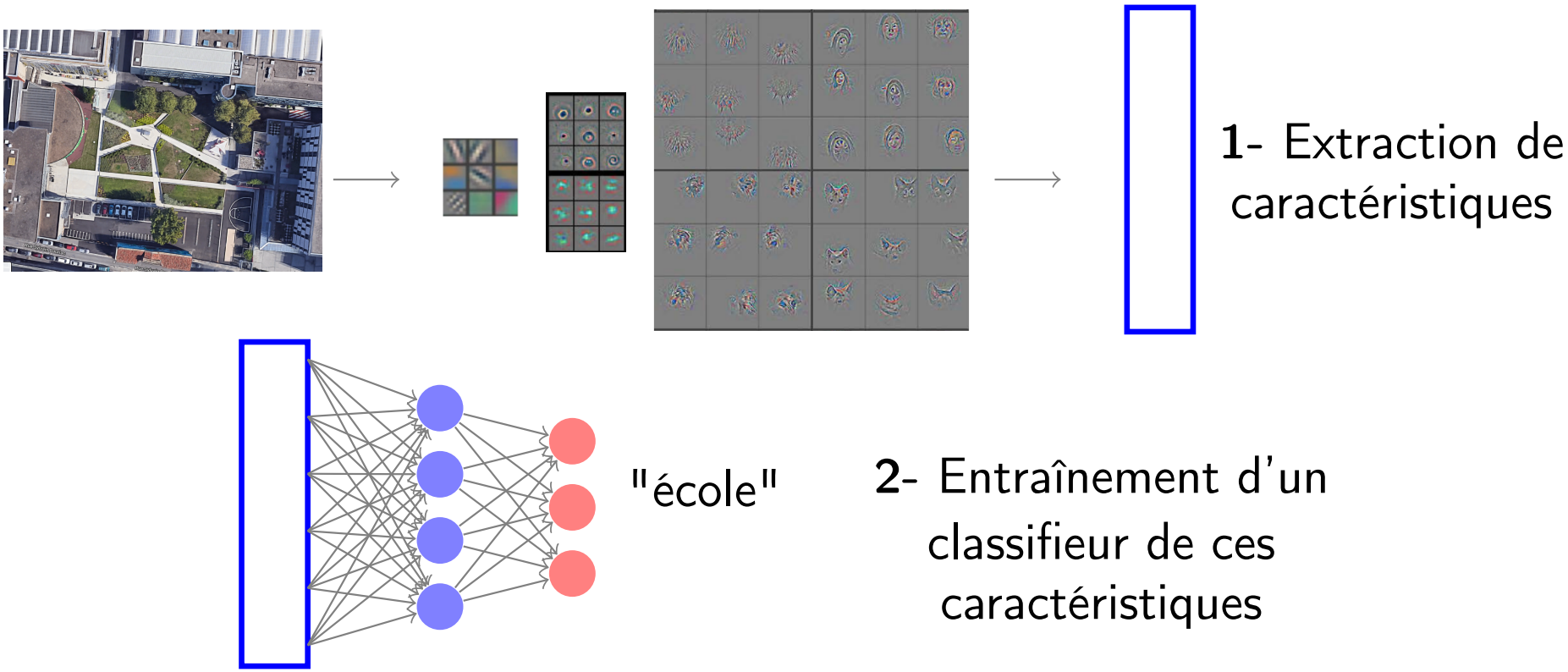
Supposons que l'on dispose d'un CNN entraîné sur une large base de données, comme ImageNet ($\approx$ 14 millions d'images).

Transfert d'apprentissage



On peut extraire la base convolutive qui agit comme extracteur de caractéristiques, et la réutiliser pour une autre tâche. C'est ce que l'on appelle le **transfert d'apprentissage** (*transfer learning*).

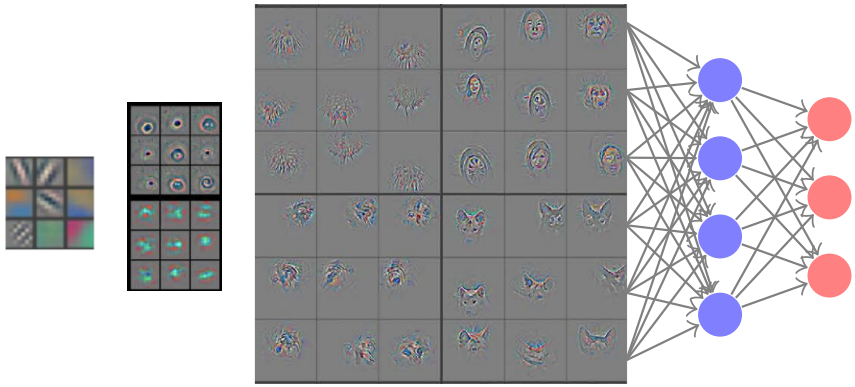
Transfert d'apprentissage "statique"



On peut utiliser un réseau pré-entraîné pour extraire les caractéristiques d'une nouvelle base de données, puis entraîner un simple classifieur de ces caractéristiques.

Transfert d'apprentissage





"école"

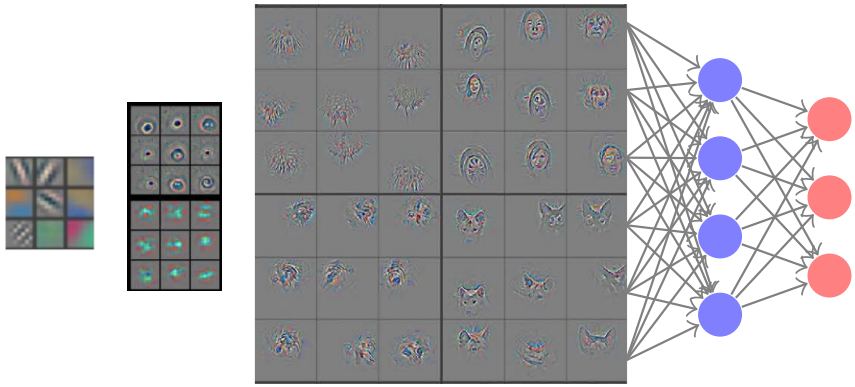
1- Gel des paramètres de l'extracteur de caractéristiques

2- Entraînement des dernières couches du classifieur

Si l'extraction de caractéristiques est incluse au classifieur, mais que ses paramètres sont bloqués, le transfert d'apprentissage supporte l'augmentation de données.

Fine-Tuning





"école"

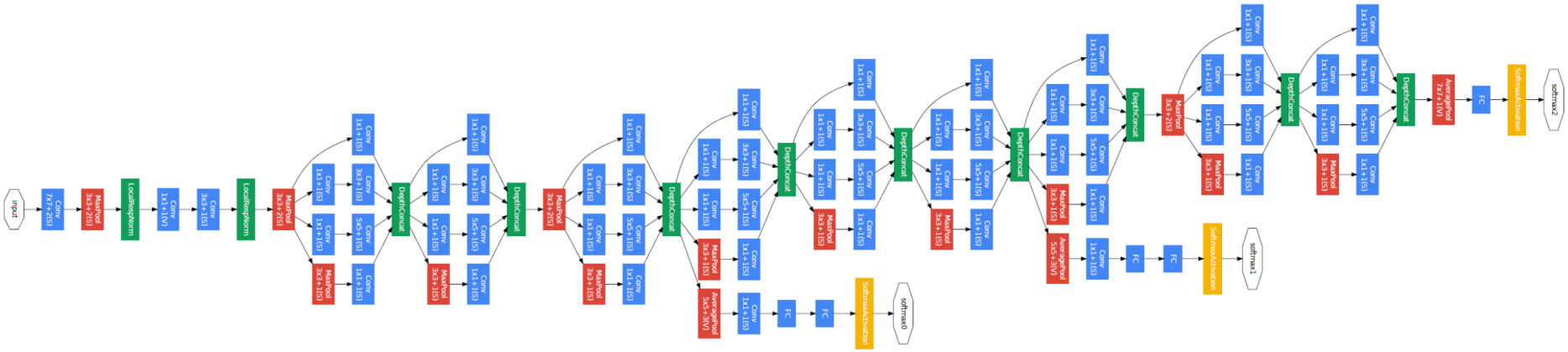
Dégel des paramètres de l'extracteur
et ré-entrainement complet du classifieur

Une fois les dernières couches du classifieur entraînées, on peut ensuite débloquer les paramètres de la base convolutive et ré-entraîner l'ensemble du réseau, pour le "spécifier" à la nouvelle tâche : c'est le *fine-tuning*.
Attention : le taux d'apprentissage utilisé doit être très petit pour ne pas risquer de détruire les filtres généraux qui avaient été obtenus lors du pré-entraînement.

Plan du cours

1. Couches de convolution
2. Architectures convolutives
3. Visualisation
4. Transfert d'apprentissage
5. Inception et ResNet
6. Segmentation

GoogLeNet (ou Inception, 2014)

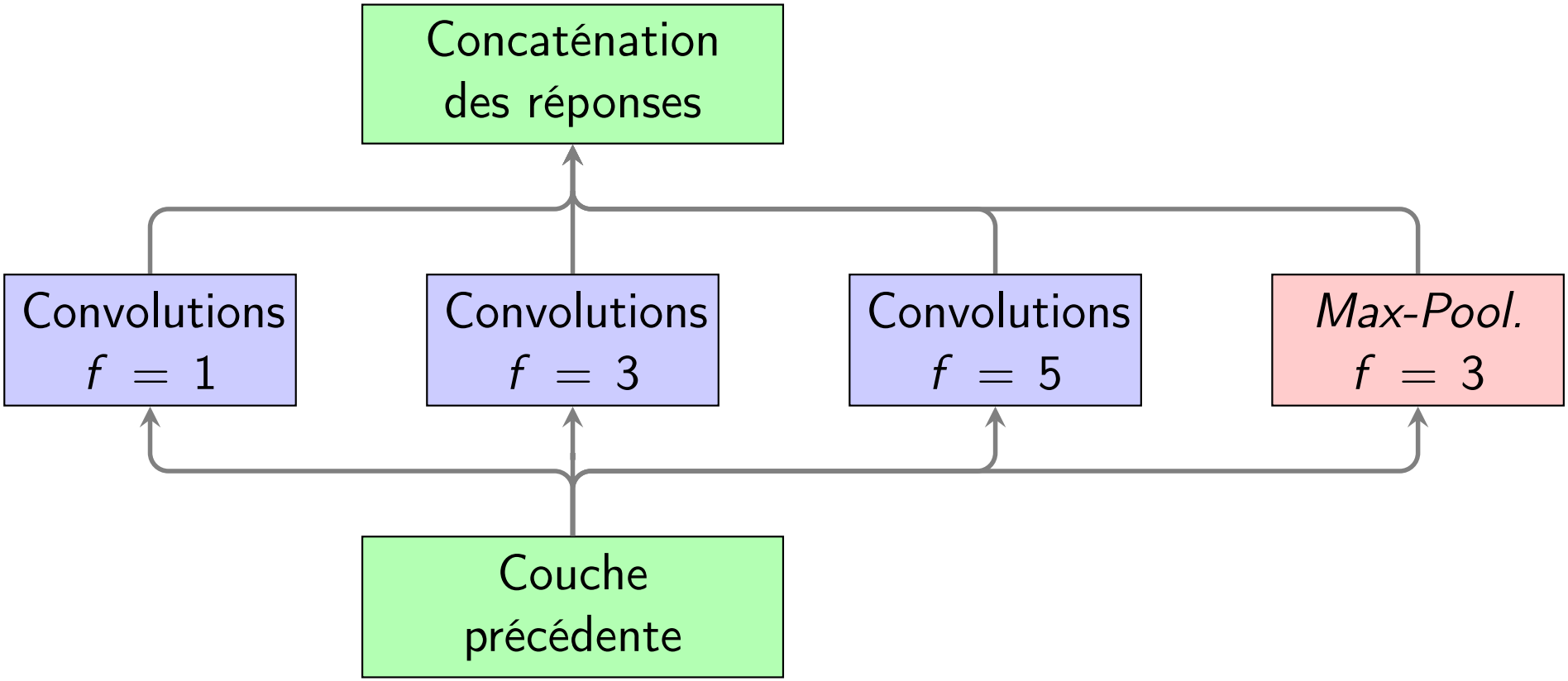


$\simeq$ 7M paramètres, 22 couches



[Szegedy et al.] Going deeper with convolutions.

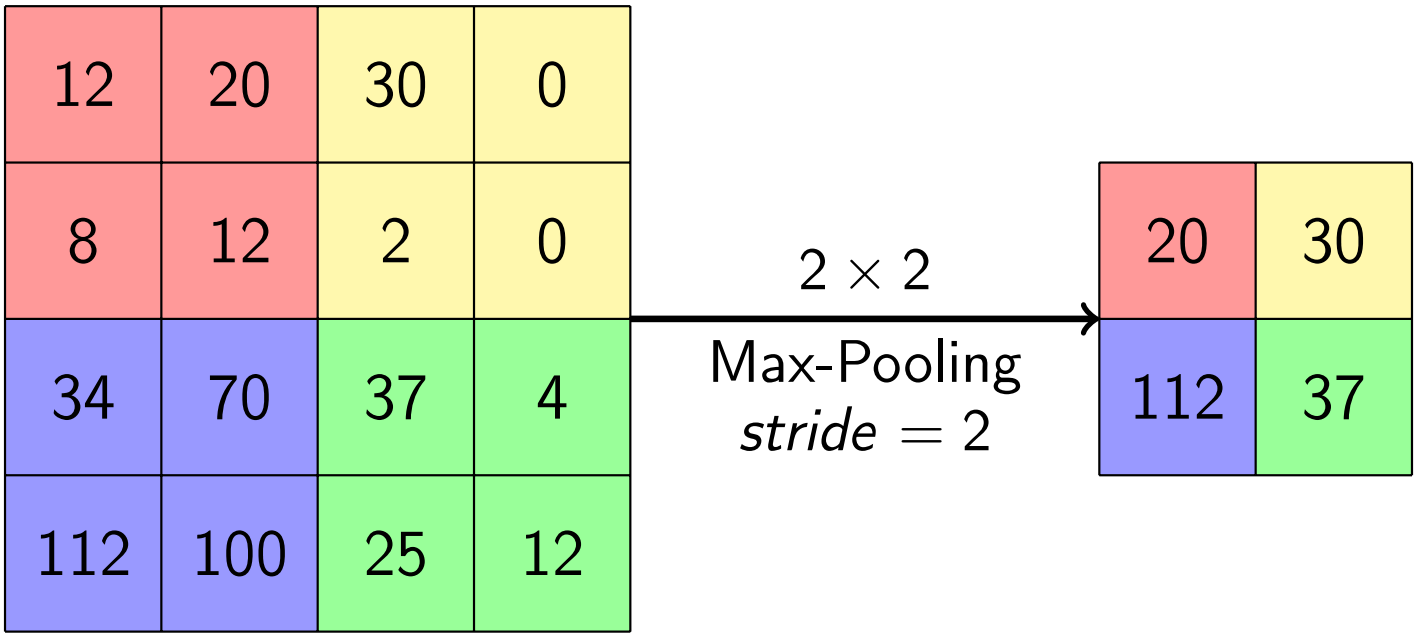
La brique de base d'Inception



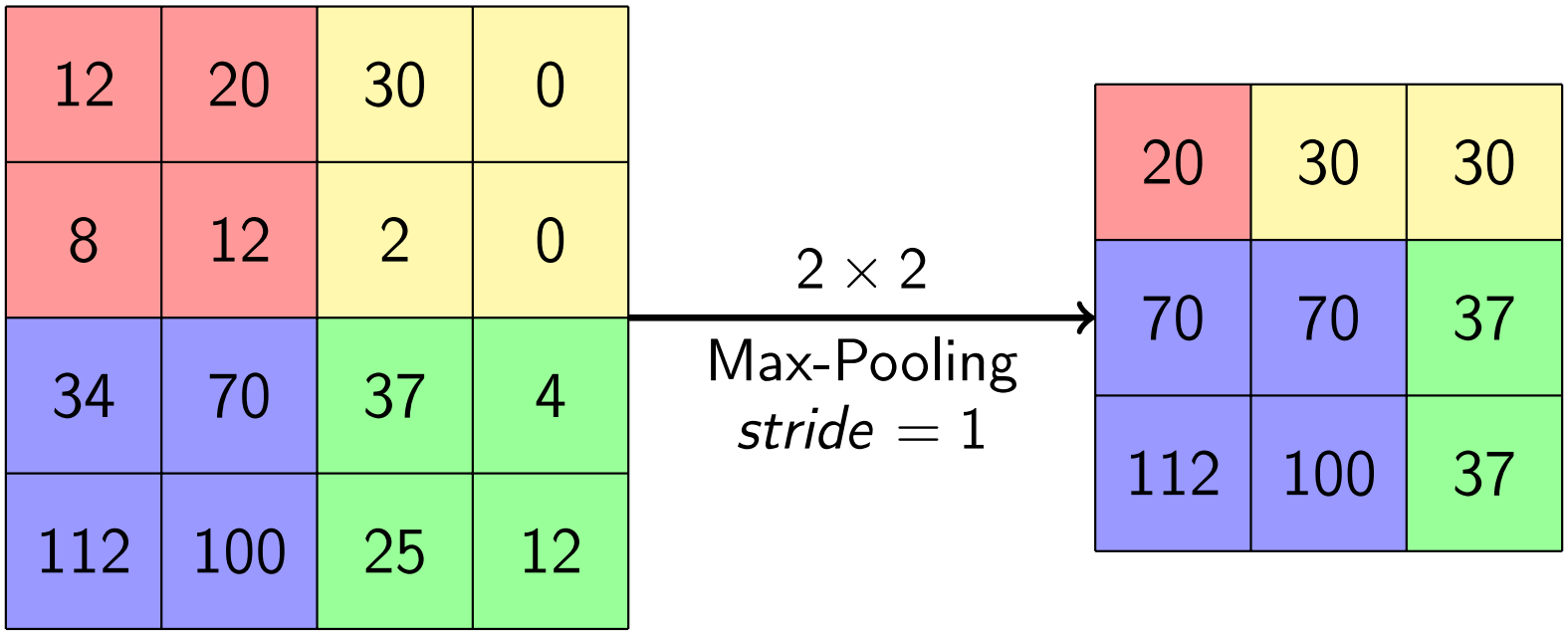
Utilisation de *padding* et d'une variante particulière du *Max-Pooling* pour conserver des tenseurs de la même dimension.

[Szegedy et al.] Going deeper with convolutions.

Couche de *Pooling* "classique"

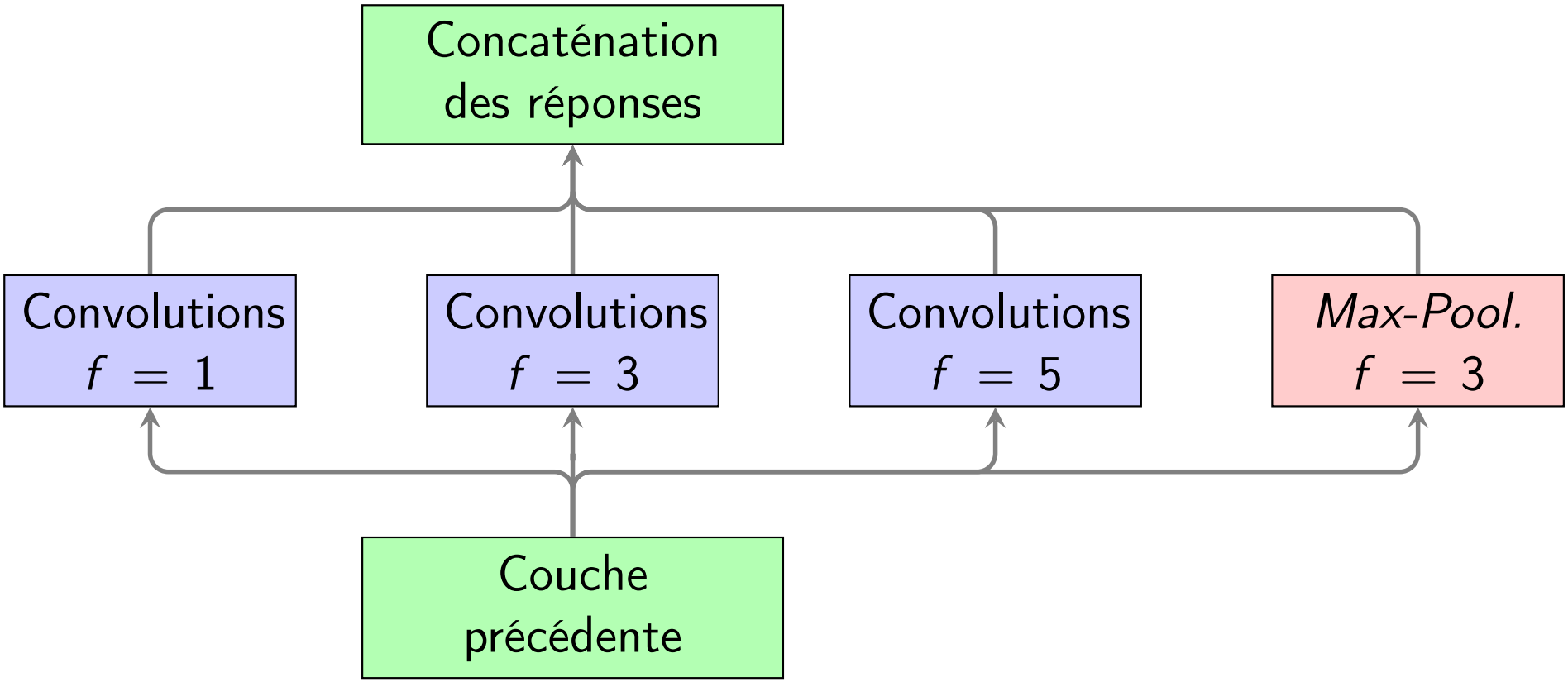


Couche de *Pooling* préservant la dimension



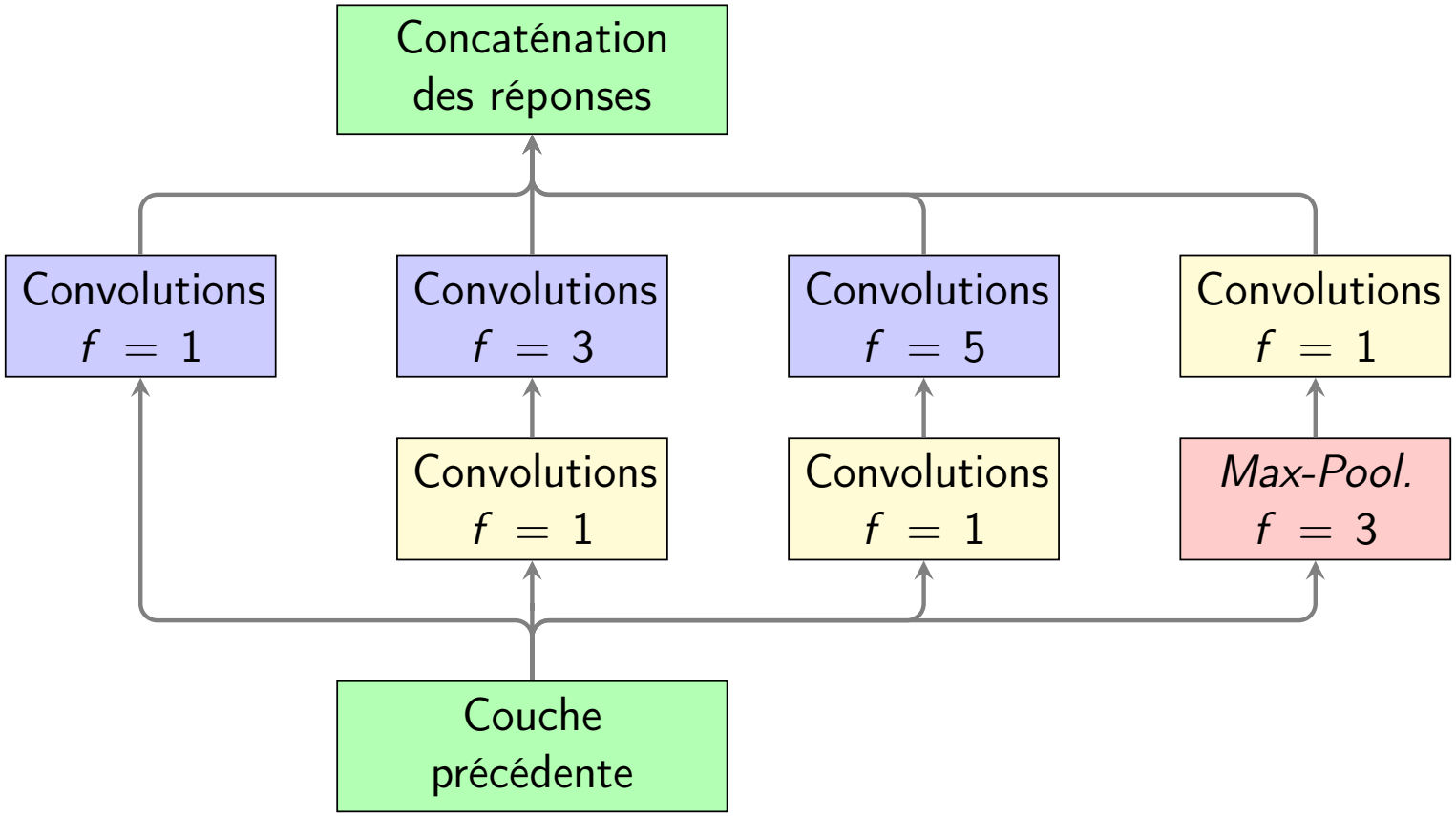
- En utilisant un *stride* de 1 et du *padding*, on peut obtenir une réponse de même dimension que l'entrée.

La brique de base d'Inception



[Szegedy et al.] Going deeper with convolutions.

La brique de base, optimisée, d'Inception



Ajout de convolutions 1×1 pour diminuer la complexité des calculs et le nombre de paramètres, mais aussi pour ajouter de la non-linéarité.

[Szegedy et al.] Going deeper with convolutions.

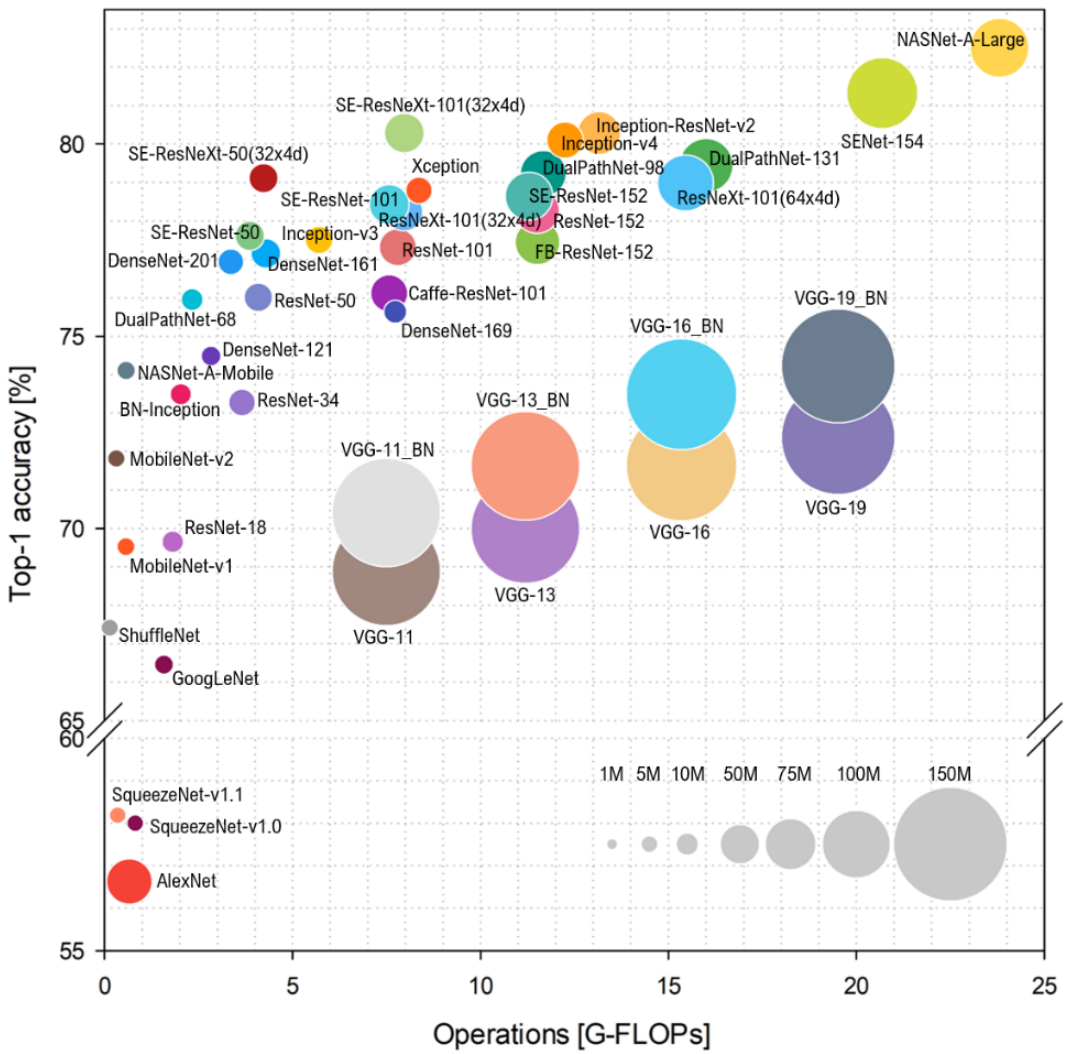
Évolutions d'Inception (v2, v3)

Nombreuses optimisations mises au point :

- Remplacement des convolutions 5×5 par 2 convolutions 3×3 successives (notion de **champ réceptif**).
- Diminution du nombre de blocs, et blocs plus "larges"
- Régularisation par *Label Smoothing*
- ...
- et plus tard encore, combinaison avec ResNet...

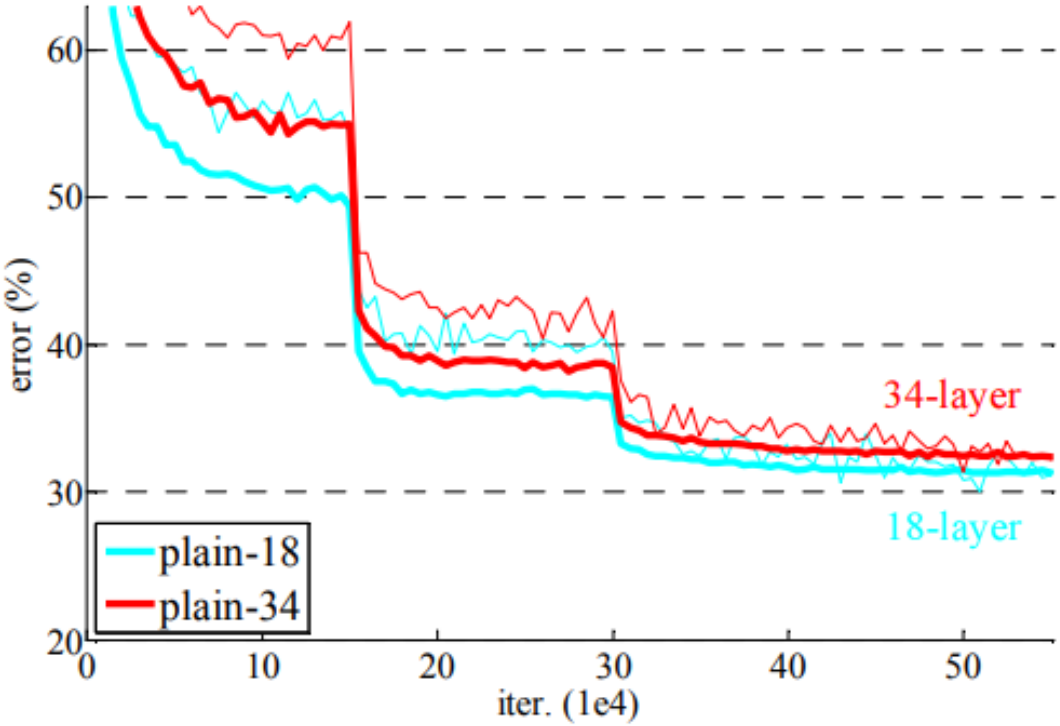
[Szegedy et al.] Rethinking the Inception Architecture for Computer Vision

Benchmark



[Bianco et al.] Benchmark Analysis of Representative Deep Neural Network Architectures

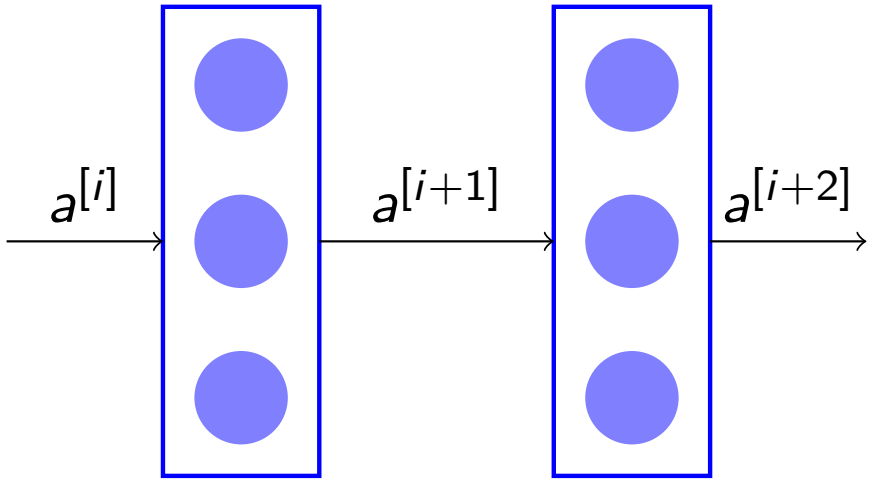
L'entraînement des réseaux profonds



Les réseaux plus profonds devraient en théorie permettre d'approximer de plus en plus efficacement l'ensemble d'apprentissage. En pratique, la difficulté à optimiser les paramètres augmente avec la profondeur.

[He et al.] Deep Residual Learning for Image Recognition

Blocs résiduels



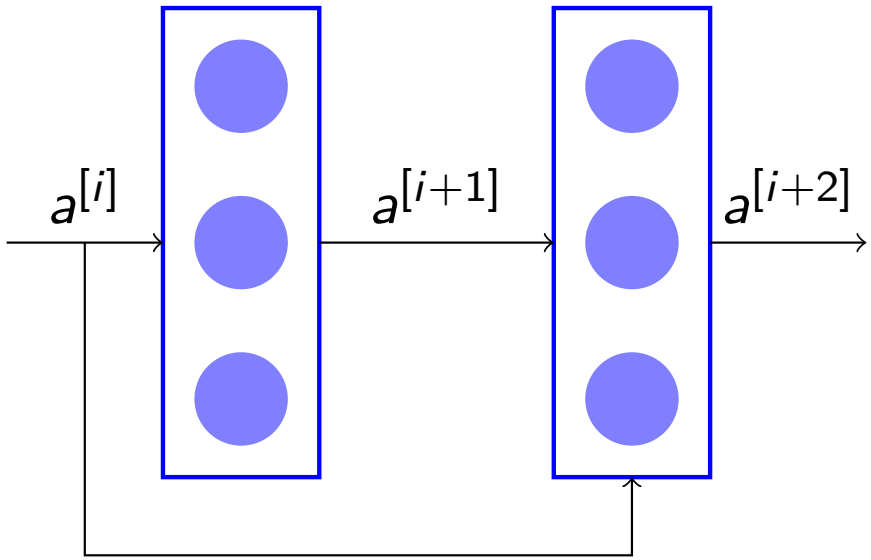
Cette vue abstrait les liaisons synaptiques entre les 2 couches $i+1$ et $i+2$. On a ici par exemple :

$$a^{[i+1]} = \texttt{ReLU}(W^{[i+1]}a^{[i]} + b^{[i+1]})$$

et

$$a^{[i+2]} = \texttt{ReLU}(W^{[i+2]}a^{[i+1]} + b^{[i+2]})$$

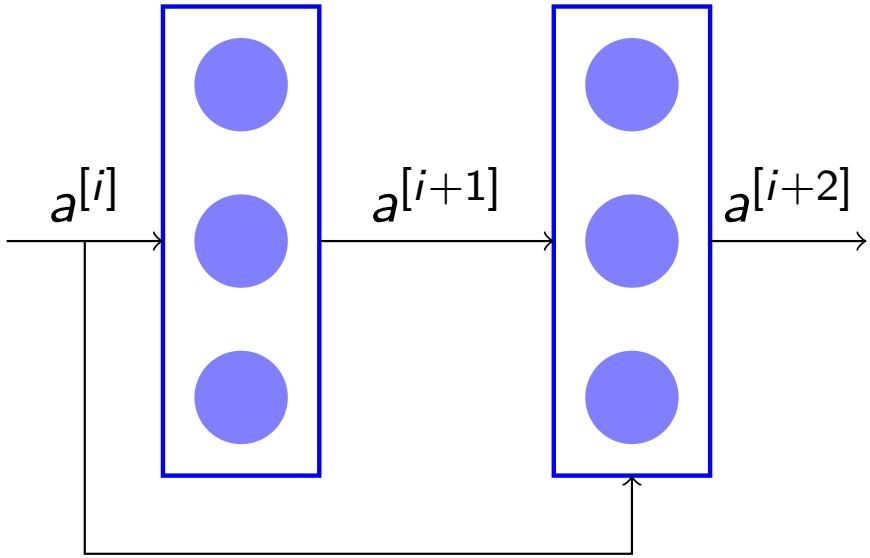
Blocs résiduels



On peut ajouter une connexion comme présenté ci-dessus (*skip connection*) pour former un **bloc résiduel**, d'équation :

$$a^{[i+2]} = \texttt{ReLU}(W^{[i+2]}a^{[i+1]} + b^{[i+2]} + a^{[i]})$$

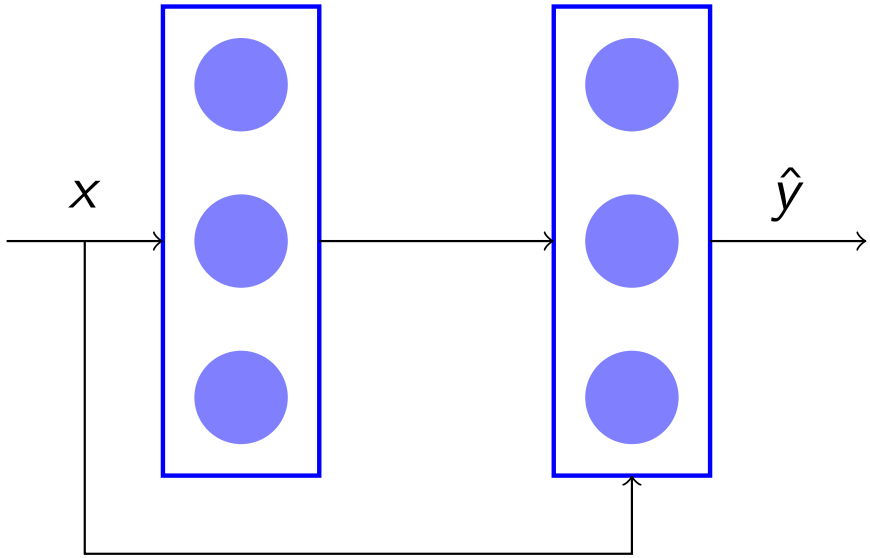
Blocs résiduels



Une condition importante est que les dimensions de $a^{[i]}$ et $a^{[i+2]}$ soient identiques !

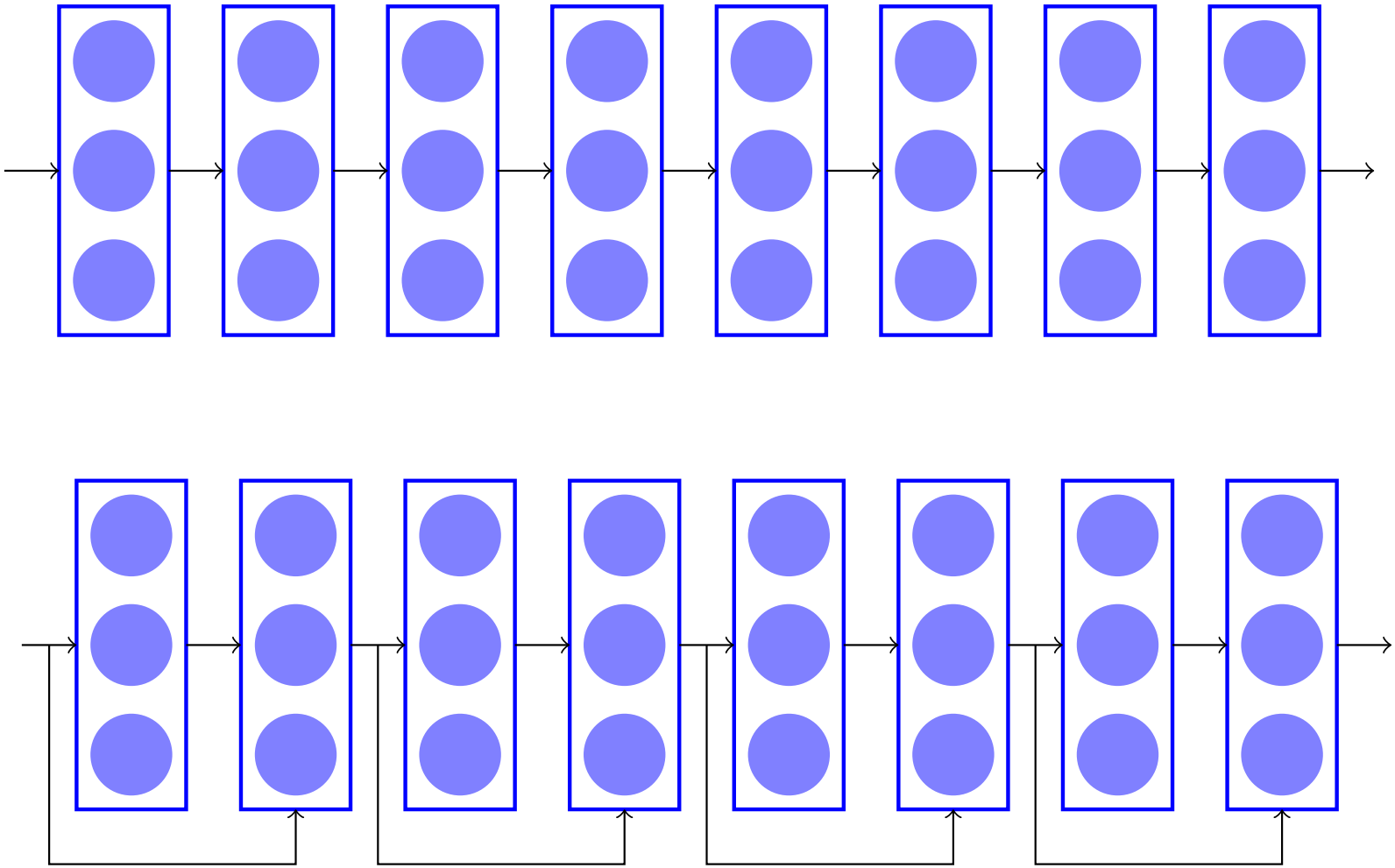
On utilise donc souvent des convolutions qui conservent la dimension (*same*) dans les réseaux résiduels.

Blocs résiduels

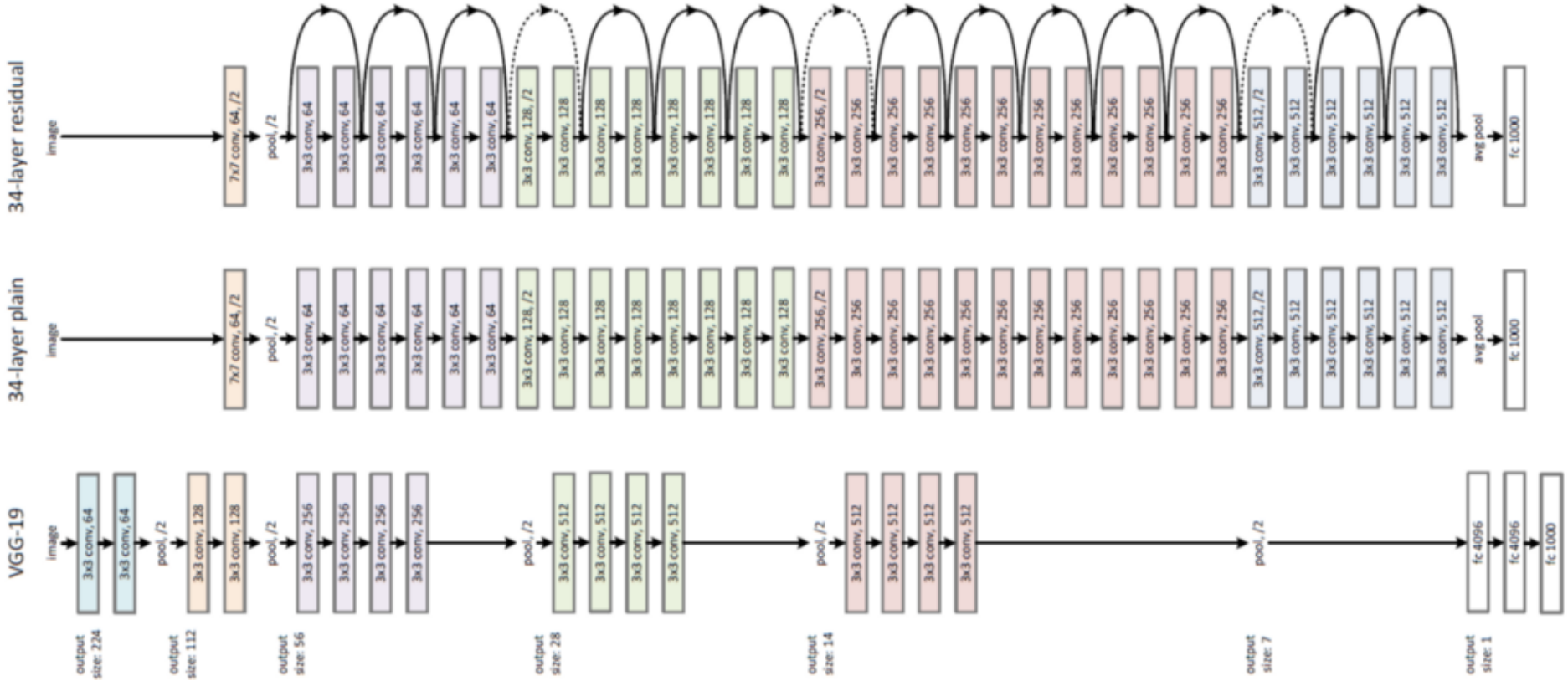


Alors que l'entraînement vise d'ordinaire à estimer une fonction F telle que $\hat{y} = F(x)$, on cherche ici la fonction telle que : $\hat{y} = F(x) + x$.
En d'autres termes, la fonction F estime le **résidu** $\hat{y} - x$ (d'où le nom de bloc résiduel).

Rendre "résiduel" un réseau de neurones

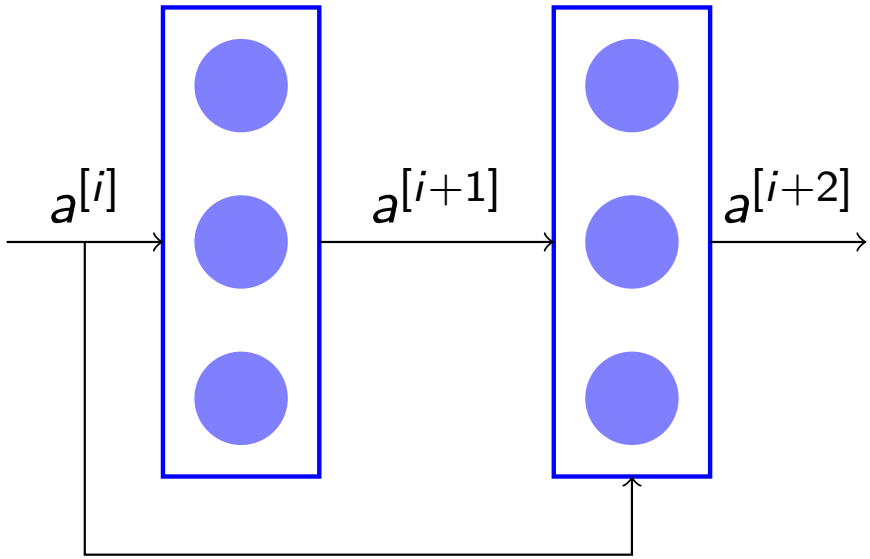


ResNet (2015)



[He et al.] Deep Residual Learning for Image Recognition

Une bonne propriété des blocs résiduels



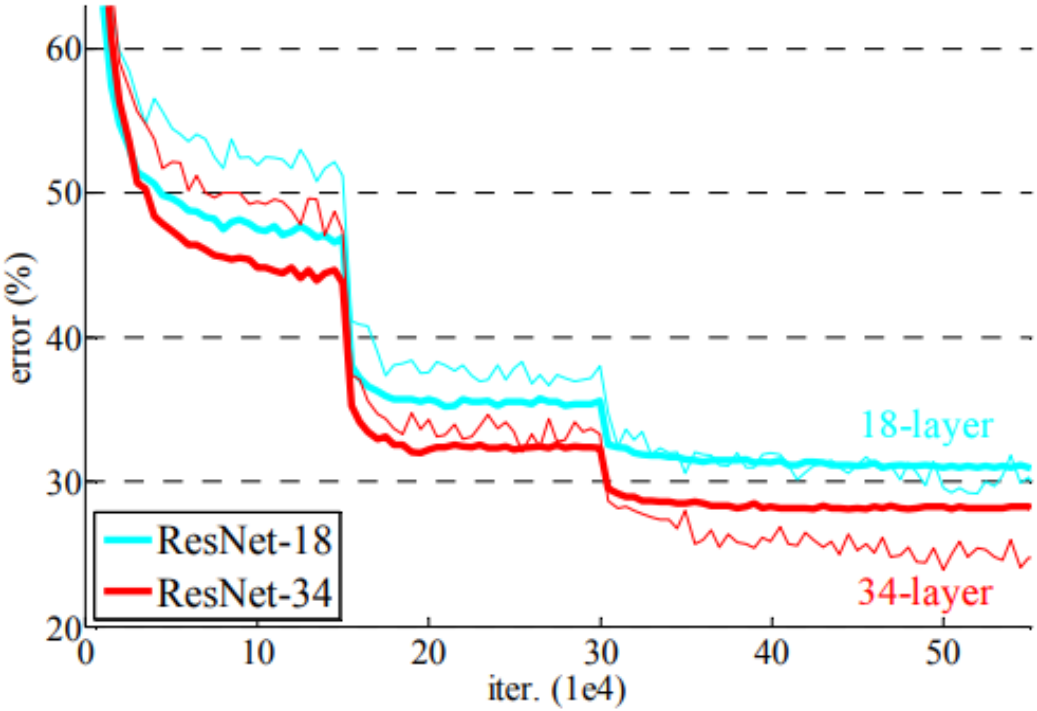
$$a^{[i+2]} = \texttt{ReLU}(W^{[i+2]}a^{[i+1]} + b^{[i+2]} + a^{[i]})$$

Si les poids synaptiques $W^{[i+2]}$ et les biais $b^{[i+2]}$ sont proches de 0, alors :

$$a^{[i+2]} \simeq \texttt{ReLU}(a^{[i]}) \simeq a^{[i]}$$

La fonction identité est facile à modéliser par un bloc résiduel. A l'initialisation, le réseau est "conditionné" à prédire l'identité.

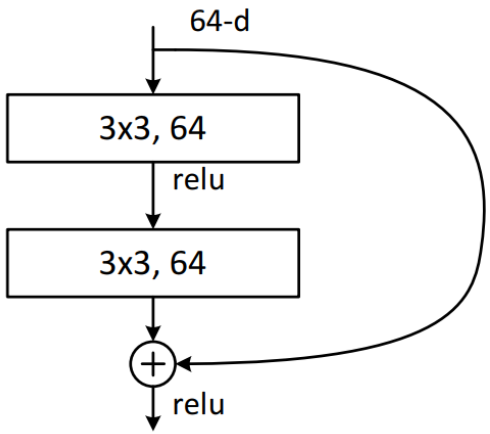
Résultats obtenus avec ResNet

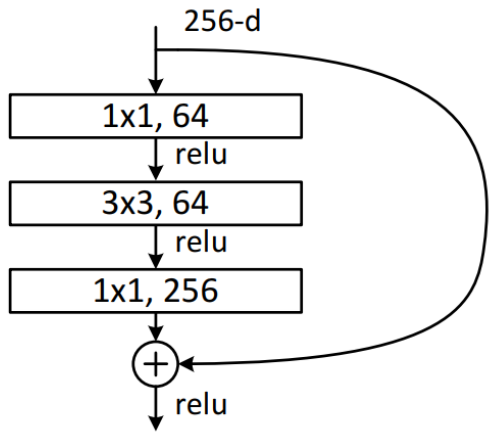


[He et al.] Deep Residual Learning for Image Recognition

ResNet (2015)

Les blocs résiduels ont permis aux auteurs d'entraîner des réseaux de plusieurs centaines de couches. Le réseau qui a remporté le *challenge* `ImageNet` en 2015 comptait d'ailleurs 152 couches.
Les problèmes de ces architectures très profondes ne sont plus liés à l'optimisation mais aux performances, ce qui a motivé les auteurs à introduire des blocs résiduels d'étranglement (*bottlenecks blocks*) :





[He et al.] Deep Residual Learning for Image Recognition

Plan du cours

1. Couches de convolution
2. Architectures convolutives
3. Visualisation
4. Transfert d'apprentissage
5. Inception et ResNet
6. Segmentation

La base de données MS COCO



plus de 300K images, et plus de 2,5M d'instances d'objets segmentés (plus de 22 heures-humain pour 1000 segmentations)

[Lin et al.] Microsoft COCO : Common Objects in Context.

Les défis de la segmentation



- Un grand nombre de prédictions : une pour chaque pixel de l'image.
- La prédiction pour un pixel donné nécessite une analyse d'information locale conjointe à une compréhension plus globale de la scène.

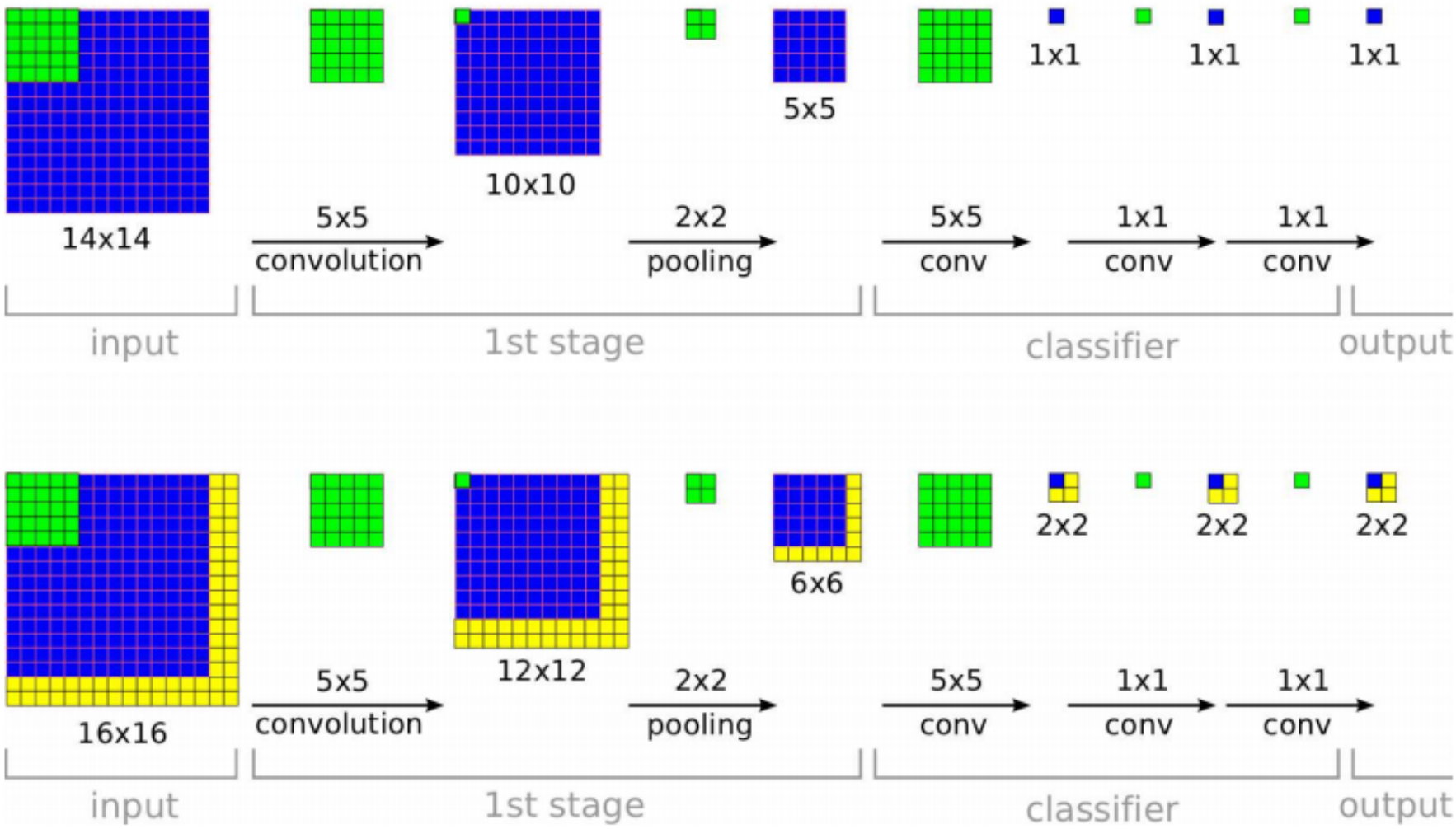
Segmentation par fenêtre glissante



Inconvénients :

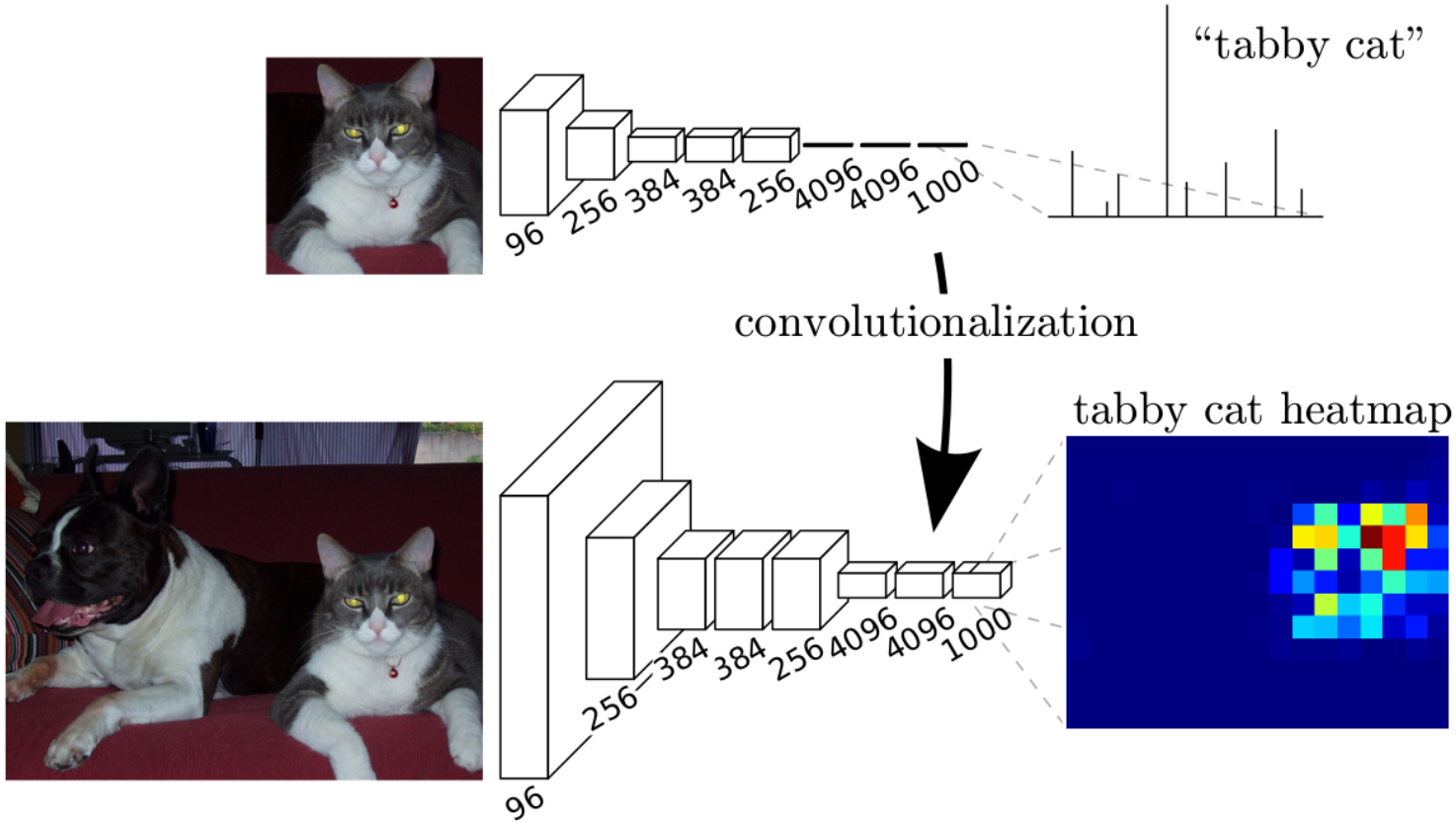
- Très coûteux.
- Pas de prise en compte de l'image globale.

Fenêtre glissante convolutionnelle



[Sermanet et al.] OverFeat : Integrated Recognition, Localization and Detection using Convolutional Networks

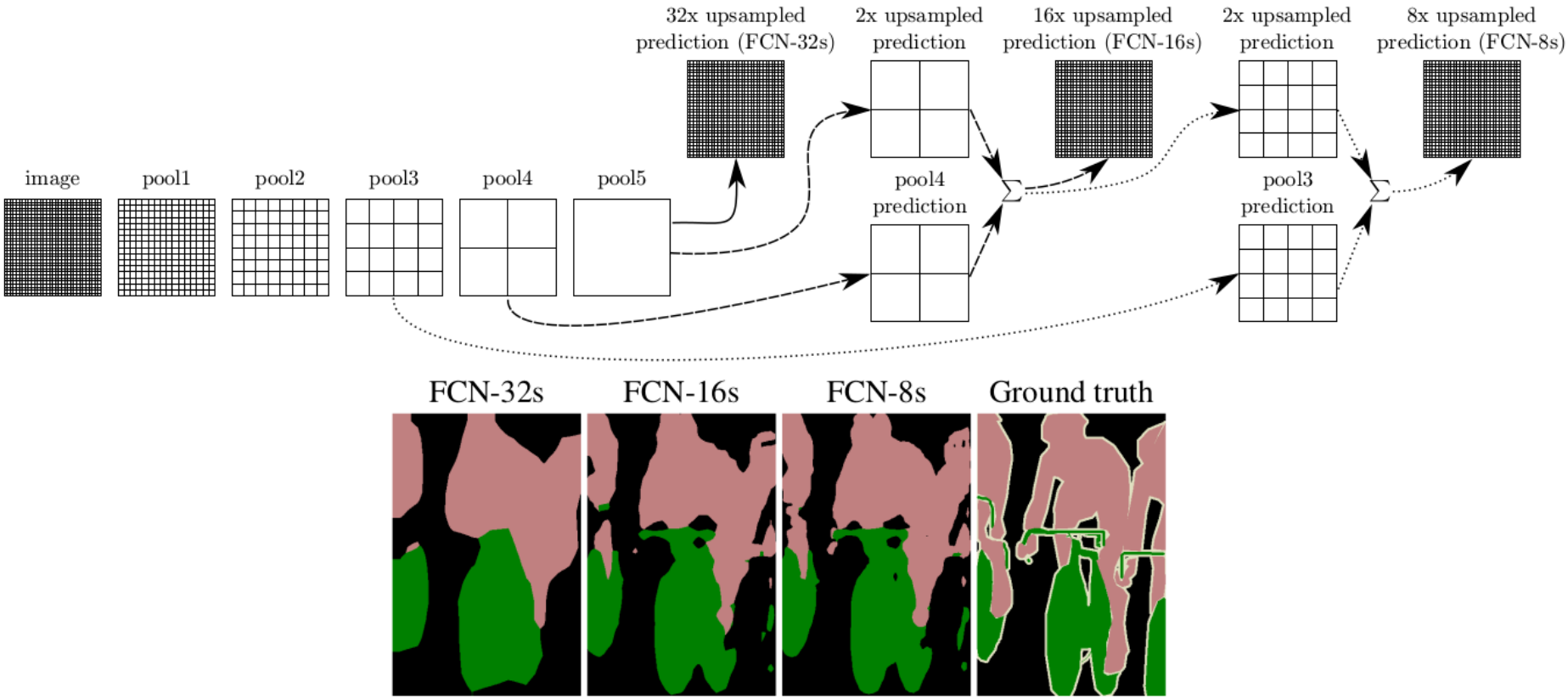
Fully Convolutional Networks (2014)



- Premier algorithme à implanter une segmentation *end-to-end*.
- Repose sur une reformulation des réseaux entièrement convolutive.
- Base VGG-16.

[Long et al.] Fully Convolutional Networks for Semantic Segmentation.

Fully Convolutional Networks (2014)



- Résultat affiné en prenant en compte des prédictions intermédiaires.

[Long et al.] Fully Convolutional Networks for Semantic Segmentation.

L'opération de déconvolution (ou convolution transposée)

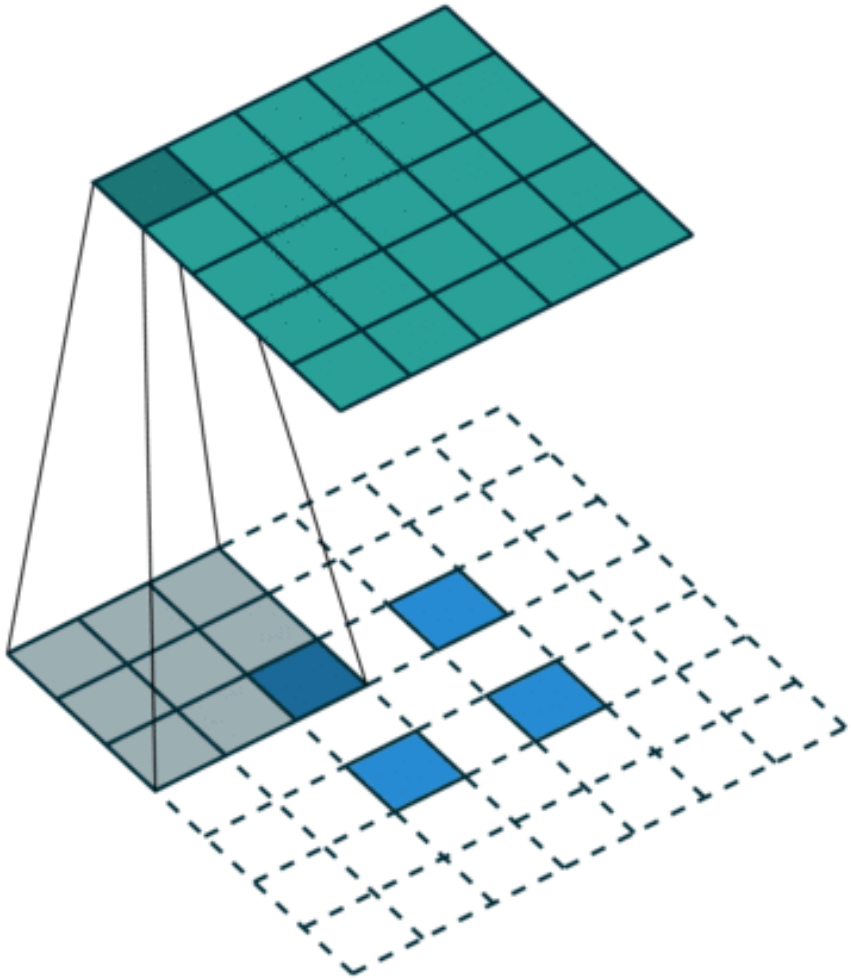
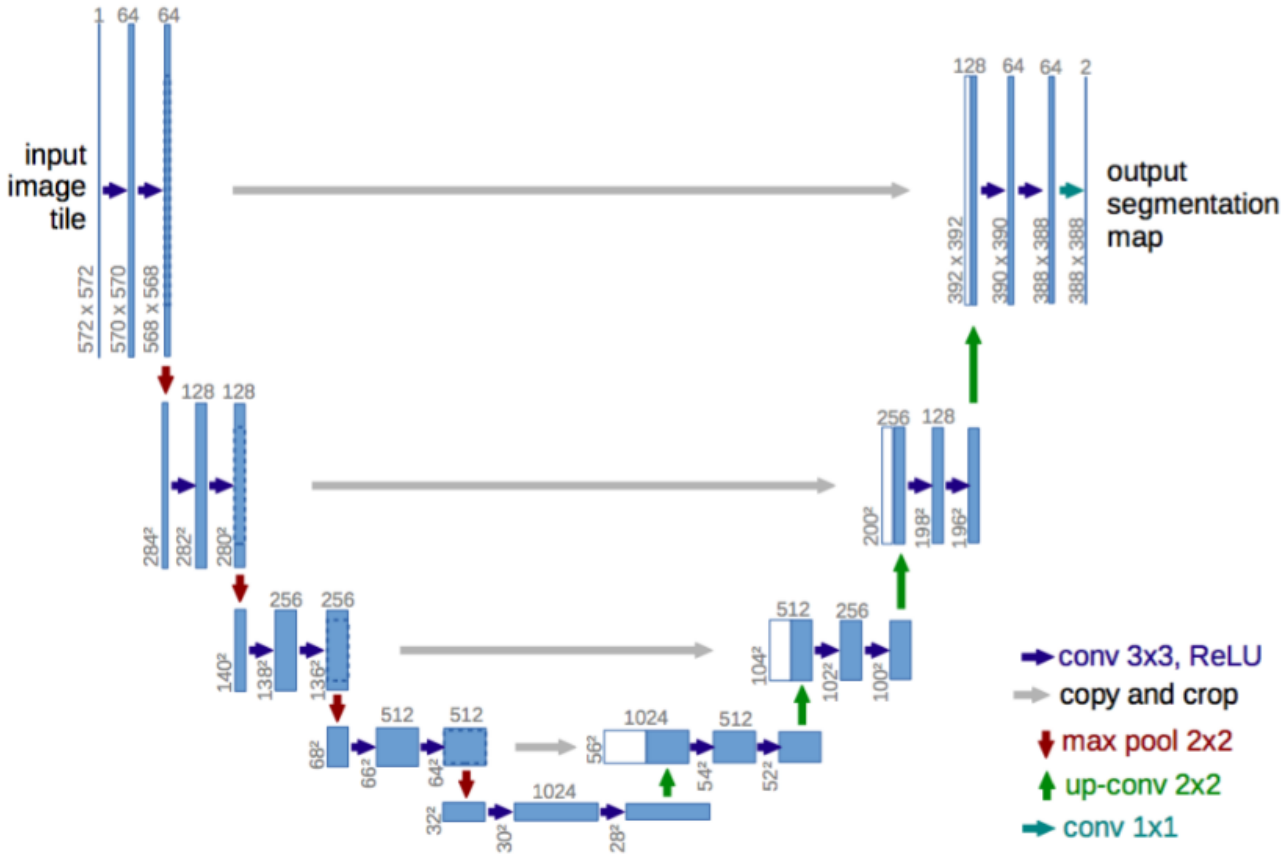


Image de `https://github.com/vdumoulin/conv_arithmetic`

Un mot sur les fonctions de coût

- Chaque pixel porte une fonction `softmax` afin de déterminer quelle classe est la plus probable.
- On utilise l'entropie croisée pour comparer la classe prédite d'un pixel et la classe réelle.
- La fonction de coût finale est la moyenne des entropies croisées sur l'ensemble des pixels de l'image.

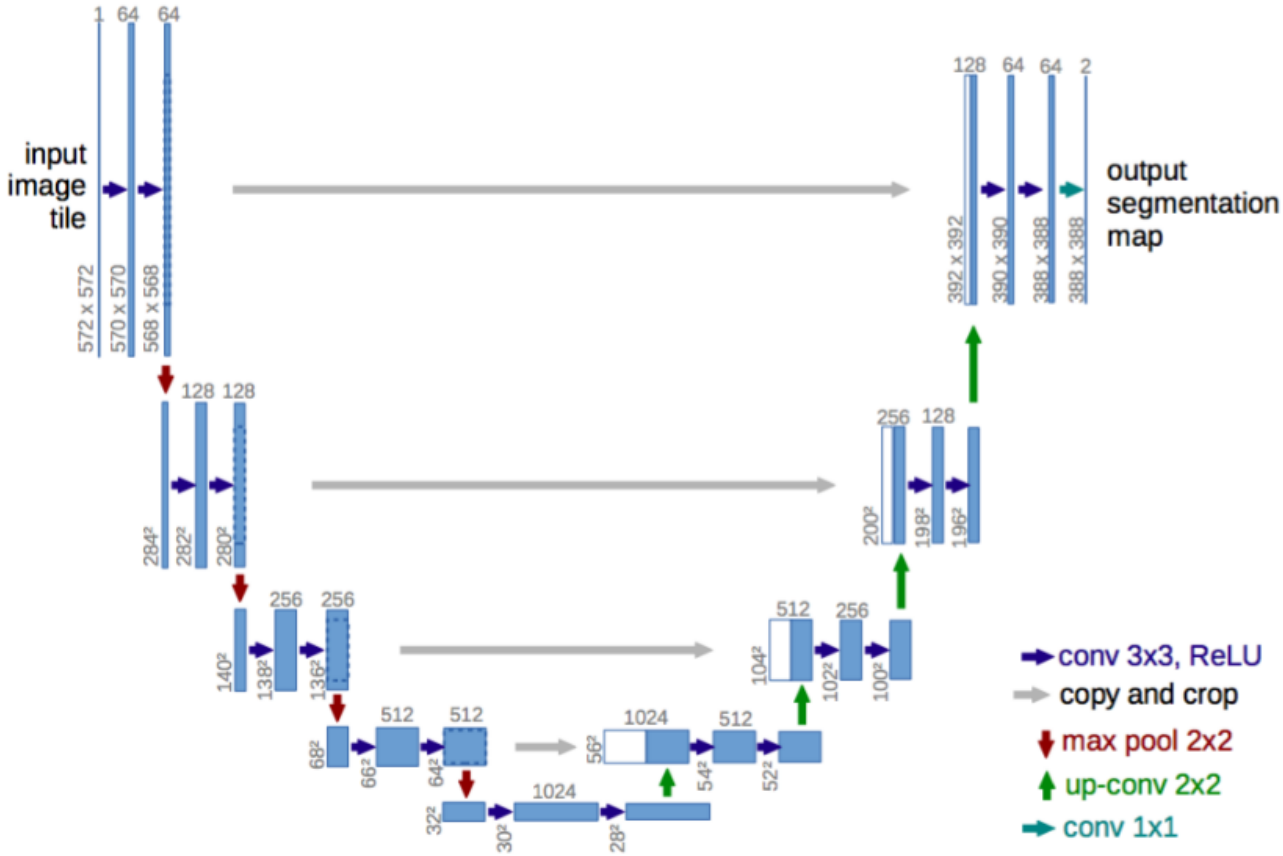
UNet (2015)



- Une évolution de FCN.
- Probablement un des réseaux les plus utilisés pour la segmentation !

[Ronneberger et al.] U-Net : Convolutional Networks for Biomedical Image Segmentation.

UNet (2015)



Architecture adaptable avec n'importe quelle base convolutive !

[Ronneberger et al.] U-Net : Convolutional Networks for Biomedical Image Segmentation.

Bilan sur la segmentation

Deux enjeux principaux :

- **Densité de la couche de sortie** : augmentation de la dimension par convolution transposée.
- **Traitement multi-échelle** : forme en U (*auto-encoder*).

Challenges :

